Code for Characterising the XY model on the lattice

1.0

Generated by Doxygen 1.15.0

1 Directory Hierarchy 1
1.1 Directories . . . 1

2 Namespace Index 3
2.1 Namespace List . . . 3

3 Hierarchical Index 3
3.1 Class Hierarchy . . . 3

4 Class Index 4
4.1 Class List . . . 4

5 File Index 4
5.1 File List . . . 4

6 Directory Documentation 5
6.1 src/lattice Directory Reference . . . 5
6.2 src/models Directory Reference . . . 6
6.3 src/simulation Directory Reference . . . 7
6.4 src Directory Reference . . . 8

7 Namespace Documentation 8
7.1 rng Namespace Reference . . . 8
7.1.1 Function Documentation . . . 9

8 Class Documentation 13
8.1 DoubleLattice Class Reference . . . 13
8.1.1 Constructor & Destructor Documentation . . . 15
8.1.2 Member Function Documentation . . . 15
8.2 IntLattice Class Reference . . . 17
8.2.1 Constructor & Destructor Documentation . . . 19
8.2.2 Member Function Documentation . . . 20
8.2.3 Member Data Documentation . . . 22
8.3 IsingModel Class Reference . . . 22
8.3.1 Constructor & Destructor Documentation . . . 24
8.3.2 Member Function Documentation . . . 24
8.4 Lattice< T > Class Template Reference . . . 28
8.4.1 Constructor & Destructor Documentation . . . 28
8.4.2 Member Function Documentation . . . 29
8.4.3 Member Data Documentation . . . 31
8.5 Model< T > Class Template Reference . . . 31
8.5.1 Detailed Description . . . 33
8.5.2 Constructor & Destructor Documentation . . . 33
8.5.3 Member Function Documentation . . . 34
8.5.4 Member Data Documentation . . . 39

8.6 ModelBase Class Reference . . . 40
8.6.1 Constructor & Destructor Documentation . . . 40
8.6.2 Member Function Documentation . . . 41
8.7 Observables Struct Reference . . . 43
8.7.1 Member Data Documentation . . . 44
8.8 PottsModel Class Reference . . . 45
8.8.1 Constructor & Destructor Documentation . . . 47
8.8.2 Member Function Documentation . . . 48
8.8.3 Member Data Documentation . . . 52
8.9 Simulation Class Reference . . . 52
8.9.1 Detailed Description . . . 54
8.9.2 Constructor & Destructor Documentation . . . 54
8.9.3 Member Function Documentation . . . 54
8.9.4 Member Data Documentation . . . 67
8.10 SimulationParameters Struct Reference . . . 68
8.10.1 Member Data Documentation . . . 68
8.11 XYModel Class Reference . . . 71
8.11.1 Constructor & Destructor Documentation . . . 73
8.11.2 Member Function Documentation . . . 73
8.11.3 Member Data Documentation . . . 78

9 File Documentation **79**
9.1 src/lattice/doubleLattice.cpp File Reference . . . 79
9.1.1 Typedef Documentation . . . 79
9.2 src/lattice/doubleLattice.h File Reference . . . 80
9.2.1 Typedef Documentation . . . 80
9.3 doubleLattice.h . . . 81
9.4 src/lattice/intLattice.cpp File Reference . . . 81
9.4.1 Typedef Documentation . . . 82
9.5 src/lattice/intLattice.h File Reference . . . 82
9.5.1 Typedef Documentation . . . 83
9.6 intLattice.h . . . 83
9.7 src/lattice/lattice.h File Reference . . . 83
9.7.1 Typedef Documentation . . . 84
9.8 lattice.h . . . 84
9.9 src/lattice/lattice.tpp File Reference . . . 85
9.10 lattice.tpp . . . 85
9.11 src/main.cpp File Reference . . . 87
9.11.1 Function Documentation . . . 87
9.12 src/models/ising.cpp File Reference . . . 88
9.12.1 Typedef Documentation . . . 88
9.13 src/models/ising.h File Reference . . . 88

9.13.1 Typedef Documentation . . . 89
9.14 ising.h . . . 89
9.15 src/models/model.h File Reference . . . 90
9.15.1 Typedef Documentation . . . 90
9.16 model.h . . . 91
9.17 src/models/model.tpp File Reference . . . 92
9.18 model.tpp . . . 92
9.19 src/models/modelBase.h File Reference . . . 93
9.20 modelBase.h . . . 93
9.21 src/models/potts.cpp File Reference . . . 94
9.21.1 Typedef Documentation . . . 94
9.22 src/models/potts.h File Reference . . . 95
9.22.1 Typedef Documentation . . . 95
9.23 potts.h . . . 96
9.24 src/models/xy.cpp File Reference . . . 96
9.24.1 Typedef Documentation . . . 97
9.25 src/models/xy.h File Reference . . . 97
9.25.1 Typedef Documentation . . . 98
9.26 xy.h . . . 98
9.27 src/random.cpp File Reference . . . 99
9.27.1 Variable Documentation . . . 99
9.28 src/random.h File Reference . . . 99
9.29 random.h . . . 100
9.30 src/simulation/observables.h File Reference . . . 101
9.30.1 Typedef Documentation . . . 102
9.31 observables.h . . . 102
9.32 src/simulation/simulation.cpp File Reference . . . 102
9.32.1 Typedef Documentation . . . 103
9.33 src/simulation/simulation.h File Reference . . . 103
9.33.1 Typedef Documentation . . . 104
9.34 simulation.h . . . 105
9.35 src/simulation/simulationParameters.h File Reference . . . 106
9.36 simulationParameters.h . . . 107

Index **109**

1 Directory Hierarchy

1.1 Directories

lattice **5**

doubleLattice.cpp **79**

doubleLattice.h 80

intLattice.cpp 81

intLattice.h 82

lattice.h 83

lattice.tpp 85

models 6

ising.cpp 88

ising.h 88

model.h 90

model.tpp 92

modelBase.h 93

potts.cpp 94

potts.h 95

xy.cpp 96

xy.h 97

simulation 7

observables.h 101

simulation.cpp 102

simulation.h 103

simulationParameters.h 106

src 8

lattice 5

doubleLattice.cpp 79

doubleLattice.h 80

intLattice.cpp 81

intLattice.h 82

lattice.h 83

lattice.tpp 85

models 6

ising.cpp 88

ising.h 88

model.h 90

model.tpp 92

modelBase.h 93

potts.cpp 94

potts.h 95

xy.cpp 96

xy.h 97

simulation 7

observables.h 101

simulation.cpp 102

simulation.h 103

simulationParameters.h 106

main.cpp 87

random.cpp 99

random.h 99

2 Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

rng 8

3 Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Lattice< T > 28

Lattice< double > 28

DoubleLattice 13

Lattice< int > 28

IntLattice 17

ModelBase 40

Model< int > 31

IsingModel 22

PottsModel 45

Model< double > 31

XYModel 71

Model< T > 31

Observables 43

Simulation 52

SimulationParameters 68

4 Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

DoubleLattice 13

IntLattice 17

IsingModel 22

Lattice< T > 28

Model< T >
Base clase for different models 31

ModelBase 40

Observables 43

PottsModel 45

Simulation
Handles the overall simulation process, including parameter parsing, model initialization, running the simulation, and writing outputs 52

SimulationParameters 68

XYModel 71

5 File Index

5.1 File List

Here is a list of all files with brief descriptions:

src/main.cpp 87

src/random.cpp 99

src/random.h 99

src/lattice/doubleLattice.cpp 79

src/lattice/doubleLattice.h 80

src/lattice/intLattice.cpp 81

src/lattice/intLattice.h 82

src/lattice/lattice.h 83

src/lattice/lattice.tpp 85

src/models/ising.cpp 88

src/models/ising.h 88

src/models/model.h 90

src/models/model.tpp 92

src/models/modelBase.h 93

src/models/potts.cpp 94

src/models/potts.h 95

src/models/xy.cpp 96

src/models/xy.h 97

src/simulation/observables.h 101

src/simulation/simulation.cpp 102

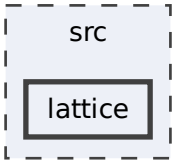
src/simulation/simulation.h 103

src/simulation/simulationParameters.h 106

6 Directory Documentation

6.1 src/lattice Directory Reference

Directory dependency graph for lattice:

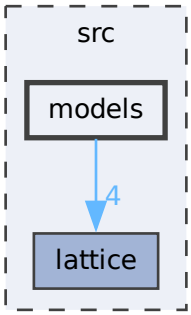


Files

- file doubleLattice.cpp
- file doubleLattice.h
- file intLattice.cpp
- file intLattice.h
- file lattice.h
- file lattice.tpp

6.2 src/models Directory Reference

Directory dependency graph for models:

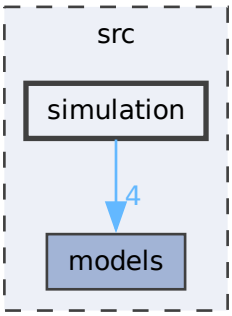


Files

- file ising.cpp
- file ising.h
- file model.h
- file model.tpp
- file modelBase.h
- file potts.cpp
- file potts.h
- file xy.cpp
- file xy.h

6.3 src/simulation Directory Reference

Directory dependency graph for simulation:

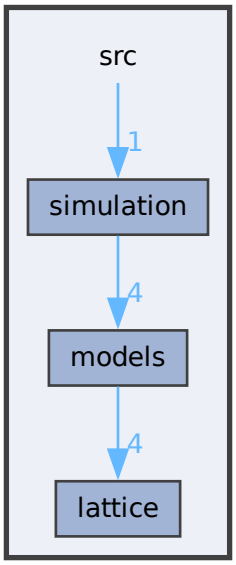


Files

- file observables.h
- file simulation.cpp
- file simulation.h
- file simulationParameters.h

6.4 src Directory Reference

Directory dependency graph for src:



Directories

- directory lattice
- directory models
- directory simulation

Files

- file main.cpp
- file random.cpp
- file random.h

7 Namespace Documentation

7.1 rng Namespace Reference

Functions

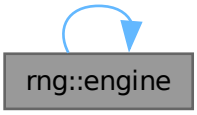
- std::mt19937 engine (0)
- void update_seed (int new_seed)
- int random_int_number (std::mt19937 &engine, int start=0, int end=1)
- double random_angle (std::mt19937 &engine)
- double random_real_number (std::mt19937 &engine)
- std::vector< int > random_int_array (std::mt19937 &engine, int length, int start=0, int end=1)
- std::vector< double > random_angle_array (std::mt19937 &engine, int length)

7.1.1 Function Documentation

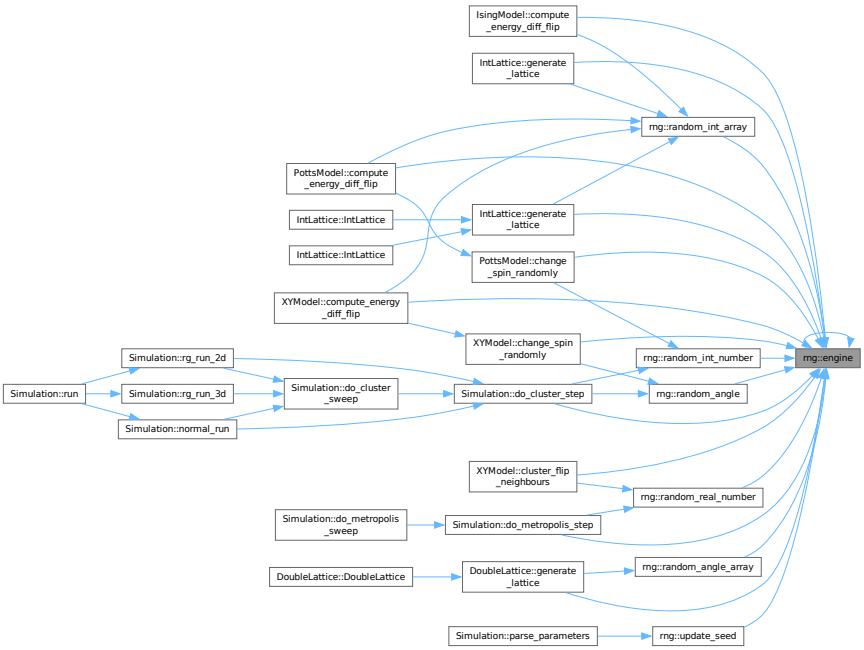
engine()

```
std::mt19937 rng::engine (
            0 ) [inline]
```

Here is the call graph for this function:



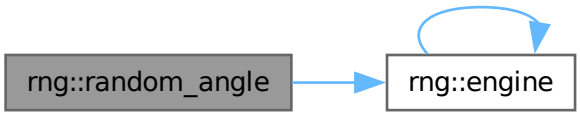
Here is the caller graph for this function:



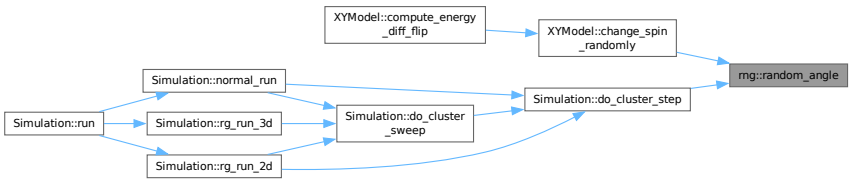
random_angle()

```
double rng::random_angle (
            std::mt19937 & engine)
```

Here is the call graph for this function:



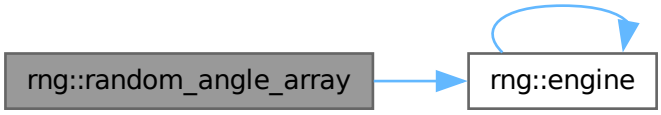
Here is the caller graph for this function:



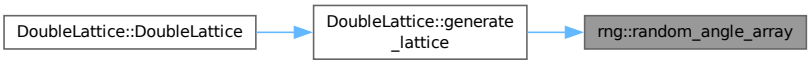
random_angle_array()

```
std::vector< double > rng::random_angle_array (
            std::mt19937 & engine,
            int length)
```

Here is the call graph for this function:



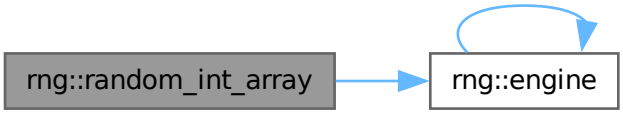
Here is the caller graph for this function:



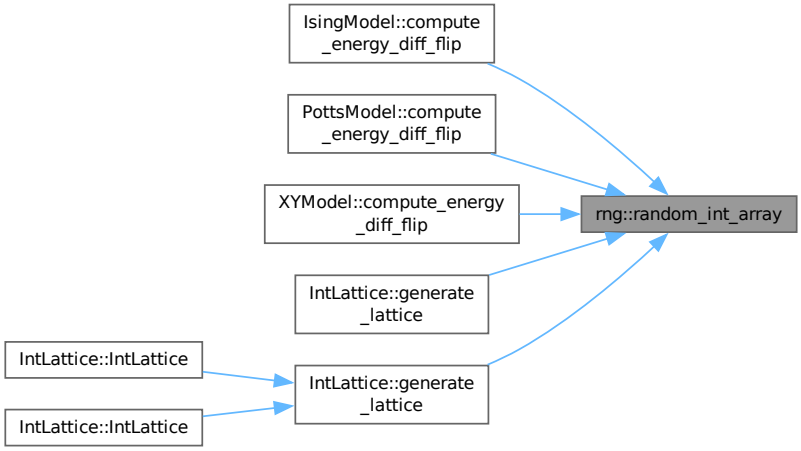
random_int_array()

```
std::vector< int > rng::random_int_array (
            std::mt19937 & engine,
            int length,
            int start = 0,
            int end = 1)
```

Here is the call graph for this function:



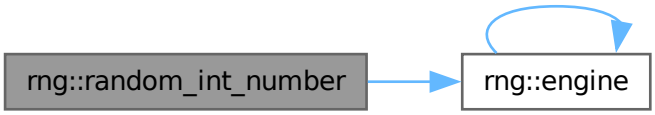
Here is the caller graph for this function:



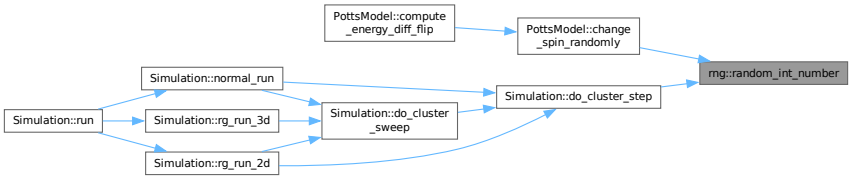
random_int_number()

```
int rng::random_int_number (
            std::mt19937 & engine,
            int start = 0,
            int end = 1)
```

Here is the call graph for this function:



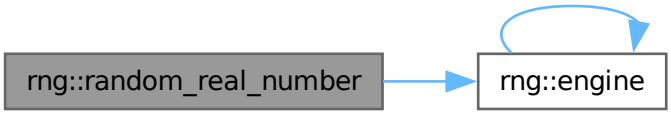
Here is the caller graph for this function:



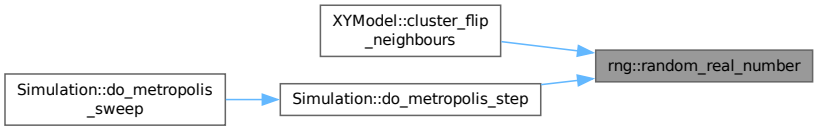
random_real_number()

```
double rng::random_real_number (
            std::mt19937 & engine)
```

Here is the call graph for this function:



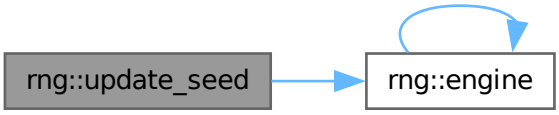
Here is the caller graph for this function:



update_seed()

```
void rng::update_seed (
            int new_seed)
```

Here is the call graph for this function:



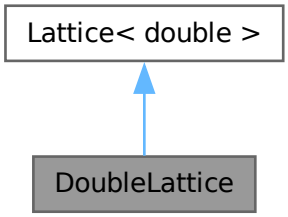
Here is the caller graph for this function:

8 Class Documentation

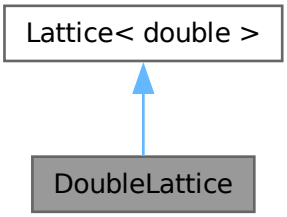
8.1 DoubleLattice Class Reference

```
#include <doubleLattice.h>
```

Inheritance diagram for DoubleLattice:



Collaboration diagram for DoubleLattice:



Public Member Functions

- DoubleLattice (const int L, const int dim)
- double & get_lattice_site (ivec indices) override
- double & get_lattice_site (int index) override
- dvec get_neighbours_array (ivec indices) override
- dvec get_neighbours_array (int index) override

Public Member Functions inherited from Lattice< double >

- virtual ~Lattice ()=default
- Lattice (int L, int dim)
- int get_1d_index (ivec indices)
- int get_lattice_total_length ()
- int get_lattice_length ()
- int get_lattice_dim ()
- int get_particle_num ()
- const std::vector< double > & get_lattice () const
- const std::vector< int > & get_neighbours_table () const

Private Member Functions

- dvec generate_lattice (int L, int dim) override

Additional Inherited Members

Protected Member Functions inherited from Lattice< double >

- ivec calculate_neighbours_table (int L, int dim)

Protected Attributes inherited from Lattice< double >

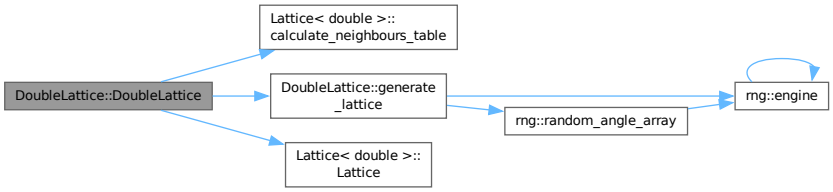
- const int lattice_dim
- const int lattice_length
- const int particle_num
- ivec neighbours_table
- std::vector< double > lattice

8.1.1 Constructor & Destructor Documentation

DoubleLattice()

```
DoubleLattice::DoubleLattice (
            const int L,
            const int dim)
```

Here is the call graph for this function:



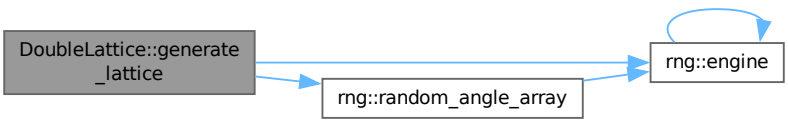
8.1.2 Member Function Documentation

generate_lattice()

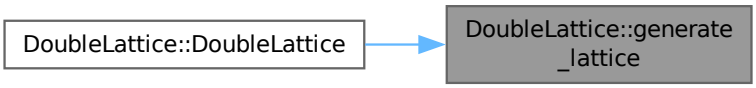
```
dvec DoubleLattice::generate_lattice (
            int L,
            int dim)  [override], [private], [virtual]
```

Implements Lattice< double >.

Here is the call graph for this function:



Here is the caller graph for this function:



get_lattice_site() [1/2]

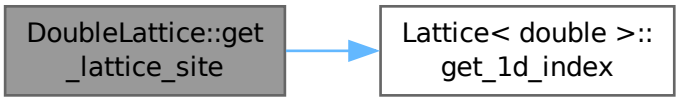
```
double & DoubleLattice::get_lattice_site (
            int index)  [override]
```

get_lattice_site() [2/2]

```
double & DoubleLattice::get_lattice_site (
            ivec indices)  [override], [virtual]
```

Implements Lattice< double >.

Here is the call graph for this function:



get_neighbours_array() [1/2]

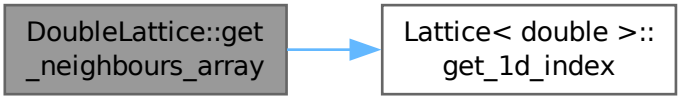
```
dvec DoubleLattice::get_neighbours_array (
            int index)  [override]
```

get_neighbours_array() [2/2]

```
dvec DoubleLattice::get_neighbours_array (
            ivec indices)  [override], [virtual]
```

Implements Lattice< double >.

Here is the call graph for this function:



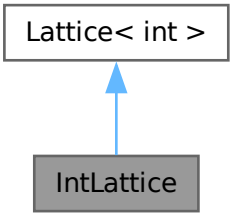
The documentation for this class was generated from the following files:

- src/lattice/doubleLattice.h
- src/lattice/doubleLattice.cpp

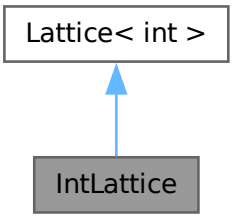
8.2 IntLattice Class Reference

```
#include <intLattice.h>
```

Inheritance diagram for IntLattice:



Collaboration diagram for IntLattice:



Public Member Functions

- IntLattice (int L, int dim, int q)
- IntLattice (int L, int dim)
- int & get_lattice_site (ivec indices) override
- int & get_lattice_site (int index) override
- ivec get_neighbours_array (ivec indices) override
- ivec get_neighbours_array (int index) override

Public Member Functions inherited from Lattice< int >

- virtual ~Lattice ()=default
- Lattice (int L, int dim)
- int get_1d_index (ivec indices)
- int get_lattice_total_length ()
- int get_lattice_length ()
- int get_lattice_dim ()
- int get_particle_num ()
- const std::vector< int > & get_lattice () const
- const std::vector< int > & get_neighbours_table () const

Private Member Functions

- ivec generate_lattice (int L, int dim, int q)
- ivec generate_lattice (int L, int dim) override

Private Attributes

- int potts_q

Additional Inherited Members

Protected Member Functions inherited from Lattice< int >

- ivec calculate_neighbours_table (int L, int dim)

Protected Attributes inherited from Lattice< int >

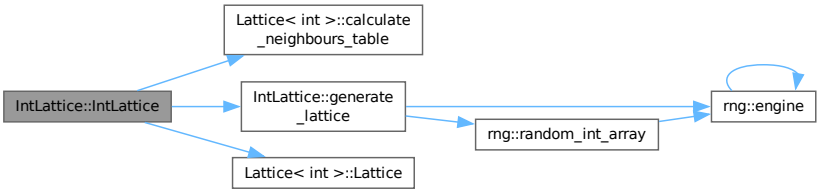
- const int lattice_dim
- const int lattice_length
- const int particle_num
- ivec neighbours_table
- std::vector< int > lattice

8.2.1 Constructor & Destructor Documentation

IntLattice() [1/2]

```
IntLattice::IntLattice (
            int L,
            int dim,
            int q)
```

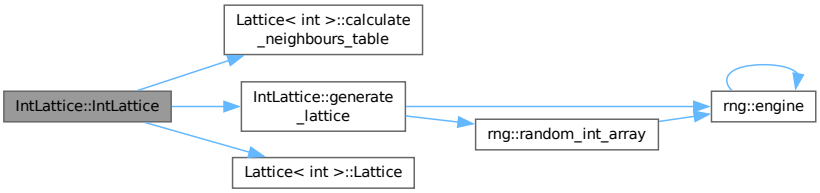
Here is the call graph for this function:



IntLattice() [2/2]

```
IntLattice::IntLattice (
            int L,
            int dim)
```

Here is the call graph for this function:



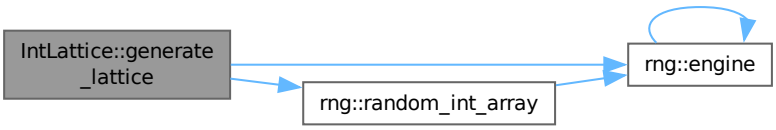
8.2.2 Member Function Documentation

generate_lattice() [1/2]

```
ivec IntLattice::generate_lattice (
            int L,
            int dim)  [override], [private], [virtual]
```

Implements Lattice< int >.

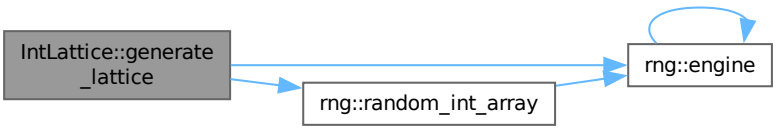
Here is the call graph for this function:



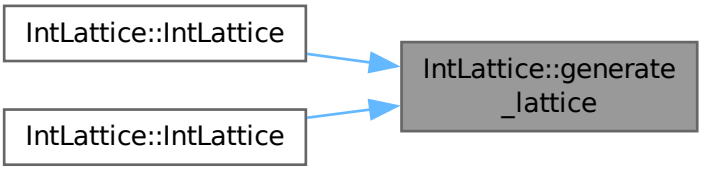
generate_lattice() [2/2]

```
ivec IntLattice::generate_lattice (
            int L,
            int dim,
            int q)  [private]
```

Here is the call graph for this function:



Here is the caller graph for this function:



get_lattice_site() [1/2]

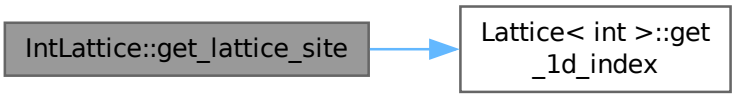
```
int & IntLattice::get_lattice_site (
            int index)  [override]
```

get_lattice_site() [2/2]

```
int & IntLattice::get_lattice_site (
            ivec indices)  [override], [virtual]
```

Implements Lattice< int >.

Here is the call graph for this function:



get_neighbours_array() [1/2]

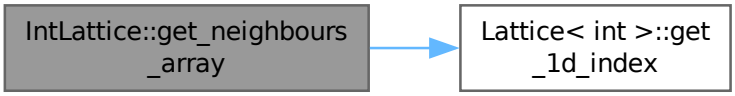
```
ivec IntLattice::get_neighbours_array (
            int index)  [override]
```

get_neighbours_array() [2/2]

```
ivec IntLattice::get_neighbours_array (
            ivec indices)  [override], [virtual]
```

Implements Lattice< int >.

Here is the call graph for this function:



8.2.3 Member Data Documentation

potts_q

```
int IntLattice::potts_q [private]
```

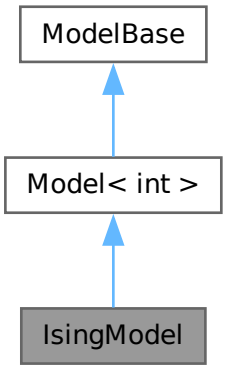
The documentation for this class was generated from the following files:

- src/lattice/intLattice.h
- src/lattice/intLattice.cpp

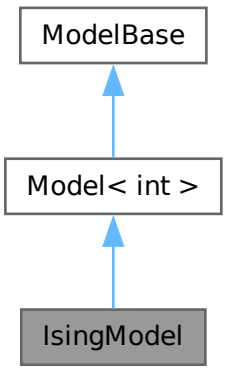
8.3 IsingModel Class Reference

```
#include <ising.h>
```

Inheritance diagram for IsingModel:



Collaboration diagram for IsingModel:



Public Member Functions

- IsingModel (double temp, double J, const int dim, const int L)
- int compute_spin_neighbours_term (int index) override

  Computes the spin term due to neighbouring spins for a given index.
- int compute_spin_neighbours_term (ivec indices) override
- int compute_spin_magnetic_term () override
- double compute_spin_magnetic_term (int dim) override

  computes the spin in the magnetic term of the hamiltonian for a given dimension
- double compute_total_energy () override
- double compute_magnetisation () override
- void change_spin_randomly (ivec indices) override

  Given a site lattice index, it randomly changes the spin at that site.
- double compute_energy_diff_flip () override

Public Member Functions inherited from Model< int >

- virtual ~Model ()=default

  Destroy the Model object.
- Model (double temp, double J)

  Default Constructor.
- double compute_average_energy (std::vector< int > energies)

  Computes the average energy of the system.
- double compute_heat_capacity (std::vector< int > energies)

  Computes the heat capacity of the system.
- double compute_single_partition_function () override

  Computes the partition function for a single configuration of the system.
- void revert_state_of_system () override

  Given an old configuration it reverts the system to that state.
- void write_lattice (HighFive::DataSet *lattice_set, hsize_t time, hsize_t N) override

  Given a DataSet it writes the current time step lattice configuration to it.
- void change_spin (int index, double spin) override
- void flip_spin (int index, double angle) override
- void cluster_flip_neighbours (int index, double direction, ivec &cluster_stack, int &spins_flipped, std::vector< uint8_t > &visited, int lattice_dim) override
- void compute_reduced_lattice (int b, size_t N, size_t L) override
- void compute_reduced_lattice_3d (int b, size_t N, size_t L) override
- double compute_rg_spin_magnetic_term (int dim) override
- double compute_rg_energy (int b, size_t N, ivec &neigh_table, int dim) override
- ivec calculate_reduced_neighbours_table (int L, int dim, int b) override

Public Member Functions inherited from ModelBase

- virtual ~ModelBase ()=default

Additional Inherited Members

Protected Attributes inherited from Model< int >

- double beta
- double J
- double temp
- std::unique_ptr< Lattice< int > > lattice_obj
- std::vector< int > old_lattice

8.3.1 Constructor & Destructor Documentation

IsingModel()

```
IsingModel::IsingModel (
            double temp,
            double J,
            const int dim,
            const int L)  [inline]
```

Here is the call graph for this function:

8.3.2 Member Function Documentation

change_spin_randomly()

```
void IsingModel::change_spin_randomly (
            ivec indices)  [override], [virtual]
```

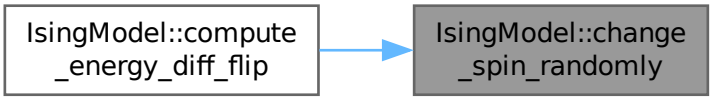
Given a site lattice index, it randomly changes the spin at that site.

Parameters

indices	array of indices where the spins are to be changed

Implements Model< int >.

Here is the caller graph for this function:

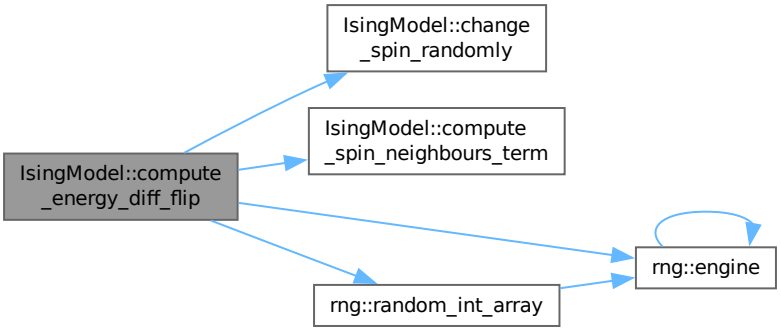


compute_energy_diff_flip()

```
double IsingModel::compute_energy_diff_flip () [override], [virtual]
```

Implements ModelBase.

Here is the call graph for this function:

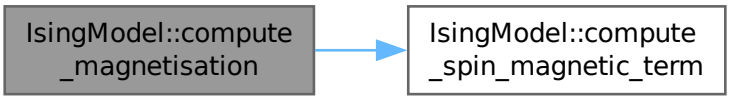


compute_magnetisation()

```
double IsingModel::compute_magnetisation () [override], [virtual]
```

Implements ModelBase.

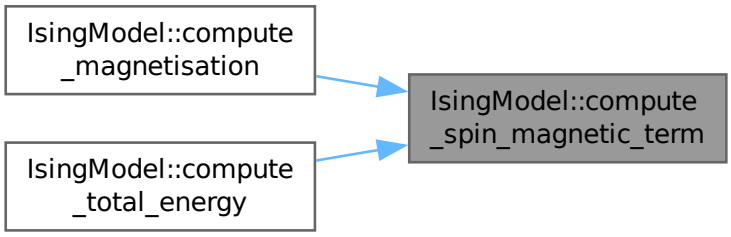
Here is the call graph for this function:



compute_spin_magnetic_term() [1/2]

```
int IsingModel::compute_spin_magnetic_term () [override]
```

Here is the caller graph for this function:



compute_spin_magnetic_term() [2/2]

```
double IsingModel::compute_spin_magnetic_term (
            int dim)  [inline], [override], [virtual]
```

computes the spin in the magnetic term of the hamiltonian for a given dimension

Parameters

dim	dimension along which to compute the magnetic term

Returns

double total spin

Implements Model< int >.

compute_spin_neighbours_term() [1/2]

```
int IsingModel::compute_spin_neighbours_term (
            int index)  [override], [virtual]
```

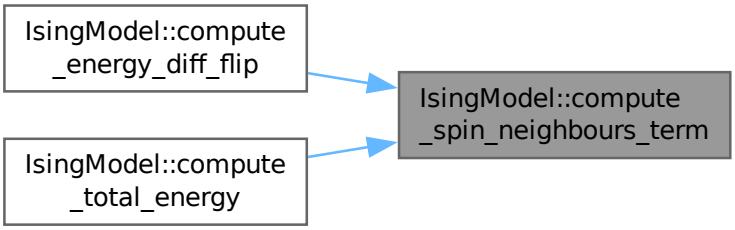
Computes the spin term due to neighbouring spins for a given index.

Returns

T total spiin

Implements Model< int >.

Here is the caller graph for this function:



compute_spin_neighbours_term() [2/2]

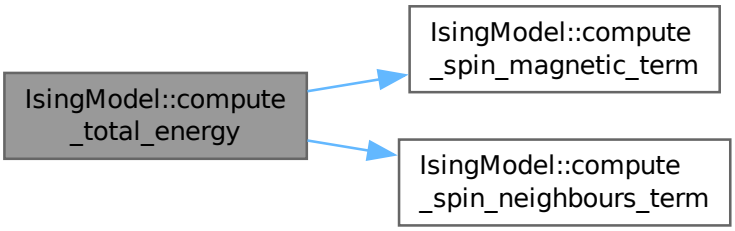
```
int IsingModel::compute_spin_neighbours_term (
            ivec indices)  [override]
```

compute_total_energy()

```
double IsingModel::compute_total_energy ()  [override], [virtual]
```

Implements ModelBase.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- src/models/ising.h
- src/models/ising.cpp

8.4 Lattice< T > Class Template Reference

```
#include <lattice.h>
```

Public Member Functions

- virtual ~Lattice ()=default
- Lattice (int L, int dim)
- int get_1d_index (ivec indices)
- int get_lattice_total_length ()
- int get_lattice_length ()
- int get_lattice_dim ()
- int get_particle_num ()
- const std::vector< T > & get_lattice () const
- std::vector< T > & get_lattice ()
- const std::vector< int > & get_neighbours_table () const
- virtual T & get_lattice_site (ivec indices)=0
- virtual T & get_lattice_site (int index)=0
- virtual std::vector< T > get_neighbours_array (ivec indices)=0
- virtual std::vector< T > get_neighbours_array (int index)=0

Protected Member Functions

- virtual std::vector< T > generate_lattice (int L, int dim)=0
- ivec calculate_neighbours_table (int L, int dim)

Protected Attributes

- const int lattice_dim
- const int lattice_length
- const int particle_num
- ivec neighbours_table
- std::vector< T > lattice

8.4.1 Constructor & Destructor Documentation

~Lattice()

```
template<typename T>
virtual Lattice< T >::~Lattice ()  [virtual], [default]
```

Lattice()

```
template<typename T>
Lattice< T >::Lattice (
            int L,
            int dim)  [inline]
```

8.4.2 Member Function Documentation

calculate_neighbours_table()

```
template<typename T>
ivec Lattice< T >::calculate_neighbours_table (
            int L,
            int dim)  [protected]
```

generate_lattice()

```
template<typename T>
virtual std::vector< T > Lattice< T >::generate_lattice (
            int L,
            int dim)  [protected], [pure virtual]
```

Implemented in DoubleLattice, and IntLattice.

get_1d_index()

```
template<typename T>
int Lattice< T >::get_1d_index (
            ivec indices)
```

get_lattice() [1/2]

```
template<typename T>
std::vector< T > & Lattice< T >::get_lattice ()  [inline]
```

get_lattice() [2/2]

```
template<typename T>
const std::vector< T > & Lattice< T >::get_lattice () const  [inline]
```

get_lattice_dim()

```
template<typename T>
int Lattice< T >::get_lattice_dim ()  [inline]
```

get_lattice_length()

```
template<typename T>
int Lattice< T >::get_lattice_length ()  [inline]
```

get_lattice_site() [1/2]

```
template<typename T>
virtual T & Lattice< T >::get_lattice_site (
            int index)  [pure virtual]
```

get_lattice_site() [2/2]

```
template<typename T>
virtual T & Lattice< T >::get_lattice_site (
            ivec indices)  [pure virtual]
```

Implemented in DoubleLattice, and IntLattice.

get_lattice_total_length()

```
template<typename T>
int Lattice< T >::get_lattice_total_length ()  [inline]
```

get_neighbours_array() [1/2]

```
template<typename T>
virtual std::vector< T > Lattice< T >::get_neighbours_array (
            int index)  [pure virtual]
```

get_neighbours_array() [2/2]

```
template<typename T>
virtual std::vector< T > Lattice< T >::get_neighbours_array (
            ivec indices)  [pure virtual]
```

Implemented in DoubleLattice, and IntLattice.

get_neighbours_table()

```
template<typename T>
const std::vector< int > & Lattice< T >::get_neighbours_table () const  [inline]
```

get_particle_num()

```
template<typename T>
int Lattice< T >::get_particle_num ()  [inline]
```

8.4.3 Member Data Documentation

lattice

```
template<typename T>
std::vector<T> Lattice< T >::lattice  [protected]
```

lattice_dim

```
template<typename T>
const int Lattice< T >::lattice_dim  [protected]
```

lattice_length

```
template<typename T>
const int Lattice< T >::lattice_length  [protected]
```

neighbours_table

```
template<typename T>
ivec Lattice< T >::neighbours_table  [protected]
```

particle_num

```
template<typename T>
const int Lattice< T >::particle_num  [protected]
```

The documentation for this class was generated from the following file:

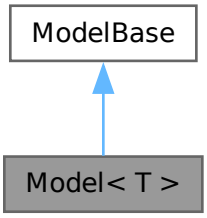
- src/lattice/lattice.h

8.5 Model< T > Class Template Reference

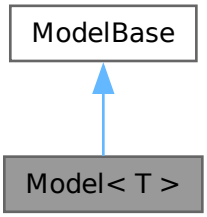
Base clase for different models.

```
#include <model.h>
```

Inheritance diagram for Model< T >:



Collaboration diagram for Model< T >:



Public Member Functions

- virtual ~Model ()=default

  Destroy the Model object.
- Model (double temp, double J)

  Default Constructor.
- double compute_average_energy (std::vector< T > energies)

  Computes the average energy of the system.
- double compute_heat_capacity (std::vector< T > energies)

  Computes the heat capacity of the system.
- double compute_single_partition_function () override

  Computes the partition function for a single configuration of the system.
- void revert_state_of_system () override

  Given an old configuration it reverts the system to that state.
- void write_lattice (HighFive::DataSet *lattice_set, hsize_t time, hsize_t N) override

  Given a DataSet it writes the current time step lattice configuration to it.
- virtual T compute_spin_neighbours_term (int index)=0

  Computes the spin term due to neighbouring spins for a given index.
- virtual T compute_spin_neighbours_term (ivec indices)=0

  Computes the spin term due to neighbouring spins for a given set of indices.
- virtual double compute_spin_magnetic_term (int dim)=0

  computes the spin in the magnetic term of the hamiltonian for a given dimension
- virtual T compute_spin_magnetic_term ()=0

  computes the spin in the magnetic term of the hamiltonian for a single dimension model
- virtual void change_spin_randomly (ivec indices)=0

  Given a site lattice index, it randomly changes the spin at that site.
- void change_spin (int index, double spin) override
- void flip_spin (int index, double angle) override
- void cluster_flip_neighbours (int index, double direction, ivec &cluster_stack, int &spins_flipped, std::vector< uint8_t > &visited, int lattice_dim) override
- void compute_reduced_lattice (int b, size_t N, size_t L) override
- void compute_reduced_lattice_3d (int b, size_t N, size_t L) override
- double compute_rg_spin_magnetic_term (int dim) override
- double compute_rg_energy (int b, size_t N, ivec &neigh_table, int dim) override
- ivec calculate_reduced_neighbours_table (int L, int dim, int b) override

Public Member Functions inherited from ModelBase

- virtual ~ModelBase ()=default
- virtual double compute_total_energy ()=0
- virtual double compute_magnetisation ()=0
- virtual double compute_energy_diff_flip ()=0

Protected Attributes

- double beta
- double J
- double temp
- std::unique_ptr< Lattice< T > > lattice_obj
- std::vector< T > old_lattice

8.5.1 Detailed Description

template<typename T>
class Model< T >

Base clase for different models.

This class provides the core functionality for simulating statistical mechanical models (e.g., Ising, Potts, XY models) on lattices. It handles energy computations, partition functions, and state management.

Template Parameters

T	data type used for lattice sites

8.5.2 Constructor & Destructor Documentation

~Model()

```
template<typename T>
virtual Model< T >::~Model ()  [virtual], [default]
```

Destroy the Model object.

Model()

```
template<typename T>
Model< T >::Model (
            double temp,
            double J)  [inline]
```

Default Constructor.

Assigns the temperature and coupling constant to their private variables

Parameters

temp	
J	

8.5.3 Member Function Documentation

calculate_reduced_neighbours_table()

```
template<typename T>
ivec Model< T >::calculate_reduced_neighbours_table (
            int L,
            int dim,
            int b)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

change_spin()

```
template<typename T>
void Model< T >::change_spin (
            int index,
            double spin)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

change_spin_randomly()

```
template<typename T>
virtual void Model< T >::change_spin_randomly (
            ivec indices)  [pure virtual]
```

Given a site lattice index, it randomly changes the spin at that site.

Parameters

indices	array of indices where the spins are to be changed

Implemented in IsingModel, PottsModel, and XYModel.

cluster_flip_neighbours()

```
template<typename T>
void Model< T >::cluster_flip_neighbours (
            int index,
            double direction,
            ivec & cluster_stack,
            int & spins_flipped,
            std::vector< uint8_t > & visited,
            int lattice_dim)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

compute_average_energy()

```
template<typename T>
double Model< T >::compute_average_energy (
            std::vector< T > energies)
```

Computes the average energy of the system.

Parameters

energies	array of energies to be averaged

Returns

double value of average energy

compute_heat_capacity()

```
template<typename T>
double Model< T >::compute_heat_capacity (
            std::vector< T > energies)
```

Computes the heat capacity of the system.

TODO include formula

Parameters

energies	array of energies

Returns

double

compute_reduced_lattice()

```
template<typename T>
void Model< T >::compute_reduced_lattice (
            int b,
            size_t N,
            size_t L)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

compute_reduced_lattice_3d()

```
template<typename T>
void Model< T >::compute_reduced_lattice_3d (
            int b,
            size_t N,
            size_t L)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

compute_rg_energy()

```
template<typename T>
double Model< T >::compute_rg_energy (
            int b,
            size_t N,
            ivec & neigh_table,
            int dim)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

compute_rg_spin_magnetic_term()

```
template<typename T>
double Model< T >::compute_rg_spin_magnetic_term (
            int dim)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

compute_single_partition_function()

```
template<typename T>
double Model< T >::compute_single_partition_function ()  [override], [virtual]
```

Computes the partition function for a single configuration of the system.

TODO INCLUDE FORMULA

Returns

double

Implements ModelBase.

compute_spin_magnetic_term() [1/2]

```
template<typename T>
virtual T Model< T >::compute_spin_magnetic_term ()  [pure virtual]
```

computes the spin in the magnetic term of the hamiltonian for a single dimension model

Returns

T total spin

compute_spin_magnetic_term() [2/2]

```
template<typename T>
virtual double Model< T >::compute_spin_magnetic_term (
            int dim)  [pure virtual]
```

computes the spin in the magnetic term of the hamiltonian for a given dimension

Parameters

dim	dimension along which to compute the magnetic term

Returns

double total spin

Implements ModelBase.

Implemented in IsingModel, PottsModel, and XYModel.

compute_spin_neighbours_term() [1/2]

```
template<typename T>
virtual T Model< T >::compute_spin_neighbours_term (
            int index)  [pure virtual]
```

Computes the spin term due to neighbouring spins for a given index.

Returns

T total spiin

Implemented in IsingModel, PottsModel, and XYModel.

compute_spin_neighbours_term() [2/2]

```
template<typename T>
virtual T Model< T >::compute_spin_neighbours_term (
            ivec indices)  [pure virtual]
```

Computes the spin term due to neighbouring spins for a given set of indices.

Returns

T total spiin

flip_spin()

```
template<typename T>
void Model< T >::flip_spin (
            int index,
            double angle)  [inline], [override], [virtual]
```

Implements ModelBase.

Reimplemented in XYModel.

revert_state_of_system()

```
template<typename T>
void Model< T >::revert_state_of_system ()  [inline], [override], [virtual]
```

Given an old configuration it reverts the system to that state.

Implements ModelBase.

write_lattice()

```
template<typename T>
void Model< T >::write_lattice (
            HighFive::DataSet * lattice_set,
            hsize_t time,
            hsize_t N)  [override], [virtual]
```

Given a DataSet it writes the current time step lattice configuration to it.

After opening hte h5 file and the dataset, this function is called to write the current lattice configuration to it at the correct time step index.

Parameters

out	*lattice_set*	a HighFive dataset object where the object is to be built
in	*time*	

Implements ModelBase.

8.5.4 Member Data Documentation

beta

```
template<typename T>
double Model< T >::beta  [protected]
```

J

```
template<typename T>
double Model< T >::J  [protected]
```

lattice_obj

```
template<typename T>
std::unique_ptr<Lattice<T> > Model< T >::lattice_obj  [protected]
```

old_lattice

```
template<typename T>
std::vector<T> Model< T >::old_lattice  [protected]
```

temp

```
template<typename T>
double Model< T >::temp  [protected]
```

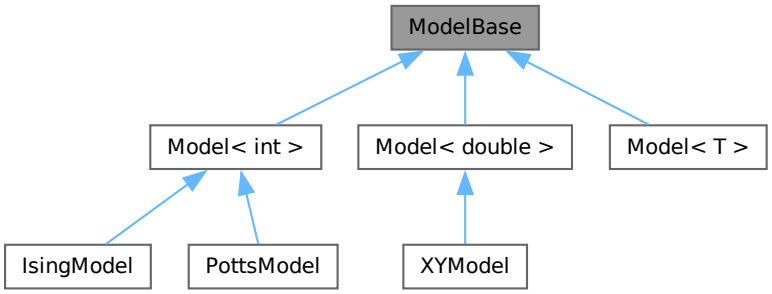
The documentation for this class was generated from the following file:

- src/models/model.h

8.6 ModelBase Class Reference

```
#include <modelBase.h>
```

Inheritance diagram for ModelBase:



Public Member Functions

- virtual ~ModelBase ()=default
- virtual double compute_single_partition_function ()=0
- virtual void revert_state_of_system ()=0
- virtual double compute_total_energy ()=0
- virtual double compute_magnetisation ()=0
- virtual double compute_energy_diff_flip ()=0
- virtual double compute_spin_magnetic_term (int dim)=0
- virtual void change_spin (int random_index, double spin)=0
- virtual void flip_spin (int index, double angle)=0
- virtual void cluster_flip_neighbours (int index, double direction, ivec &cluster_stack, int &spins_flipped, std↩ ::vector< uint8_t > &visited, int lattice_dim)=0
- virtual void write_lattice (HighFive::DataSet *lattice_set, hsize_t time, hsize_t N)=0
- virtual void compute_reduced_lattice (int b, size_t N, size_t L)=0
- virtual void compute_reduced_lattice_3d (int b, size_t N, size_t L)=0
- virtual double compute_rg_spin_magnetic_term (int dim)=0
- virtual double compute_rg_energy (int b, size_t N, ivec &neigh_table, int dim)=0
- virtual ivec calculate_reduced_neighbours_table (int L, int dim, int b)=0

8.6.1 Constructor & Destructor Documentation

~ModelBase()

```
virtual ModelBase::~ModelBase () [virtual], [default]
```

8.6.2 Member Function Documentation

calculate_reduced_neighbours_table()

```
virtual ivec ModelBase::calculate_reduced_neighbours_table (
            int L,
            int dim,
            int b)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

change_spin()

```
virtual void ModelBase::change_spin (
            int random_index,
            double spin)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

cluster_flip_neighbours()

```
virtual void ModelBase::cluster_flip_neighbours (
            int index,
            double direction,
            ivec & cluster_stack,
            int & spins_flipped,
            std::vector< uint8_t > & visited,
            int lattice_dim)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

compute_energy_diff_flip()

```
virtual double ModelBase::compute_energy_diff_flip ()  [pure virtual]
```

Implemented in IsingModel, PottsModel, and XYModel.

compute_magnetisation()

```
virtual double ModelBase::compute_magnetisation ()  [pure virtual]
```

Implemented in IsingModel, PottsModel, and XYModel.

compute_reduced_lattice()

```
virtual void ModelBase::compute_reduced_lattice (
            int b,
            size_t N,
            size_t L)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

compute_reduced_lattice_3d()

```
virtual void ModelBase::compute_reduced_lattice_3d (
            int b,
            size_t N,
            size_t L)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

compute_rg_energy()

```
virtual double ModelBase::compute_rg_energy (
            int b,
            size_t N,
            ivec & neigh_table,
            int dim)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

compute_rg_spin_magnetic_term()

```
virtual double ModelBase::compute_rg_spin_magnetic_term (
            int dim)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

compute_single_partition_function()

```
virtual double ModelBase::compute_single_partition_function ()  [pure virtual]
```

Implemented in Model< T >, Model< double >, and Model< int >.

compute_spin_magnetic_term()

```
virtual double ModelBase::compute_spin_magnetic_term (
            int dim)  [pure virtual]
```

Implemented in IsingModel, Model< T >, Model< double >, Model< int >, PottsModel, and XYModel.

compute_total_energy()

```
virtual double ModelBase::compute_total_energy ()  [pure virtual]
```

Implemented in IsingModel, PottsModel, and XYModel.

flip_spin()

```
virtual void ModelBase::flip_spin (
            int index,
            double angle)  [pure virtual]
```

Implemented in Model< T >, Model< double >, Model< int >, and XYModel.

revert_state_of_system()

```
virtual void ModelBase::revert_state_of_system ()  [pure virtual]
```

Implemented in Model< T >, Model< double >, and Model< int >.

write_lattice()

```
virtual void ModelBase::write_lattice (
            HighFive::DataSet * lattice_set,
            hsize_t time,
            hsize_t N)  [pure virtual]
```

Implemented in Model< T >, Model< double >, and Model< int >.

The documentation for this class was generated from the following file:

- src/models/modelBase.h

8.7 Observables Struct Reference

```
#include <observables.h>
```

Public Attributes

- dvec energy_array
- dvec x_magnetisation
- dvec y_magnetisation
- double susceptibility
- double correlation_length
- dvec correlation_function_array
- dvec average_cluster_size
- dvec rg_x_magnetisation1
- dvec rg_y_magnetisation1
- dvec rg_x_magnetisation2
- dvec rg_y_magnetisation2
- dvec rg_x_magnetisation3
- dvec rg_y_magnetisation3
- dvec rg_energy1
- dvec rg_energy2
- dvec rg_energy3

8.7.1 Member Data Documentation

average_cluster_size

```
dvec Observables::average_cluster_size
```

correlation_function_array

```
dvec Observables::correlation_function_array
```

correlation_length

```
double Observables::correlation_length
```

energy_array

```
dvec Observables::energy_array
```

rg_energy1

```
dvec Observables::rg_energy1
```

rg_energy2

```
dvec Observables::rg_energy2
```

rg_energy3

```
dvec Observables::rg_energy3
```

rg_x_magnetisation1

```
dvec Observables::rg_x_magnetisation1
```

rg_x_magnetisation2

```
dvec Observables::rg_x_magnetisation2
```

rg_x_magnetisation3

```
dvec Observables::rg_x_magnetisation3
```

rg_y_magnetisation1

```
dvec Observables::rg_y_magnetisation1
```

rg_y_magnetisation2

```
dvec Observables::rg_y_magnetisation2
```

rg_y_magnetisation3

```
dvec Observables::rg_y_magnetisation3
```

susceptibility

```
double Observables::susceptibility
```

x_magnetisation

```
dvec Observables::x_magnetisation
```

y_magnetisation

```
dvec Observables::y_magnetisation
```

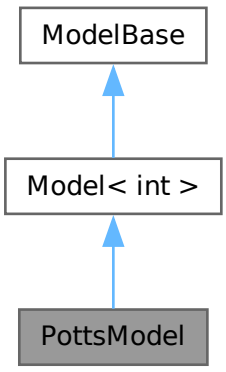
The documentation for this struct was generated from the following file:

- src/simulation/observables.h

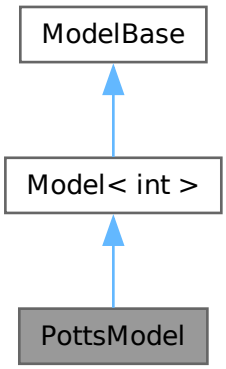
8.8 PottsModel Class Reference

```
#include <potts.h>
```

Inheritance diagram for PottsModel:



Collaboration diagram for PottsModel:



Public Member Functions

- PottsModel (double temp, double J, int dim, int L, int q)
- int compute_spin_neighbours_term (int index) override

  Computes the spin term due to neighbouring spins for a given index.
- int compute_spin_neighbours_term (ivec indices) override
- double compute_spin_magnetic_term (int dim) override

  computes the spin in the magnetic term of the hamiltonian for a given dimension
- int compute_spin_magnetic_term () override
- double compute_total_energy () override
- double compute_magnetisation () override
- void change_spin_randomly (ivec indices) override

  Given a site lattice index, it randomly changes the spin at that site.
- double compute_energy_diff_flip () override

Public Member Functions inherited from Model< int >

- virtual ~Model ()=default

  Destroy the Model object.
- Model (double temp, double J)

  Default Constructor.
- double compute_average_energy (std::vector< int > energies)

  Computes the average energy of the system.
- double compute_heat_capacity (std::vector< int > energies)

  Computes the heat capacity of the system.
- double compute_single_partition_function () override

  Computes the partition function for a single configuration of the system.
- void revert_state_of_system () override

  Given an old configuration it reverts the system to that state.
- void write_lattice (HighFive::DataSet *lattice_set, hsize_t time, hsize_t N) override

  Given a DataSet it writes the current time step lattice configuration to it.

- void change_spin (int index, double spin) override
- void flip_spin (int index, double angle) override
- void cluster_flip_neighbours (int index, double direction, ivec &cluster_stack, int &spins_flipped, std::vector< uint8_t > &visited, int lattice_dim) override
- void compute_reduced_lattice (int b, size_t N, size_t L) override
- void compute_reduced_lattice_3d (int b, size_t N, size_t L) override
- double compute_rg_spin_magnetic_term (int dim) override
- double compute_rg_energy (int b, size_t N, ivec &neigh_table, int dim) override
- ivec calculate_reduced_neighbours_table (int L, int dim, int b) override

Public Member Functions inherited from ModelBase

- virtual ~ModelBase ()=default

Private Member Functions

- int delta_function (int a, int b)

Private Attributes

- dvec vec_H
- double H
- int q

Additional Inherited Members

Protected Attributes inherited from Model< int >

- double beta
- double J
- double temp
- std::unique_ptr< Lattice< int > > lattice_obj
- std::vector< int > old_lattice

8.8.1 Constructor & Destructor Documentation

PottsModel()

```
PottsModel::PottsModel (
            double temp,
            double J,
            int dim,
            int L,
            int q)  [inline]
```

Here is the call graph for this function:

8.8.2 Member Function Documentation

change_spin_randomly()

```
void PottsModel::change_spin_randomly (
            ivec indices)  [override], [virtual]
```

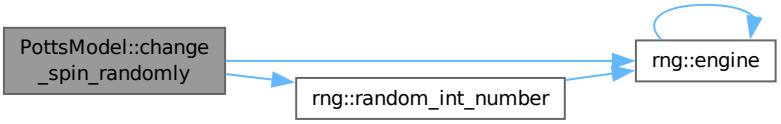
Given a site lattice index, it randomly changes the spin at that site.

Parameters

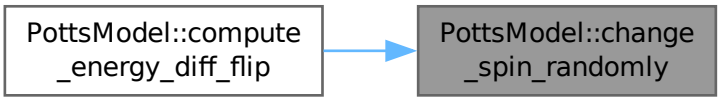
indices	array of indices where the spins are to be changed

Implements Model< int >.

Here is the call graph for this function:



Here is the caller graph for this function:

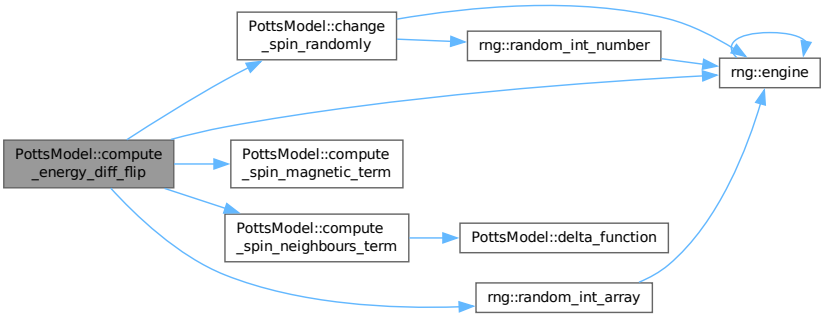


compute_energy_diff_flip()

```
double PottsModel::compute_energy_diff_flip ()  [override], [virtual]
```

Implements ModelBase.

Here is the call graph for this function:



compute_magnetisation()

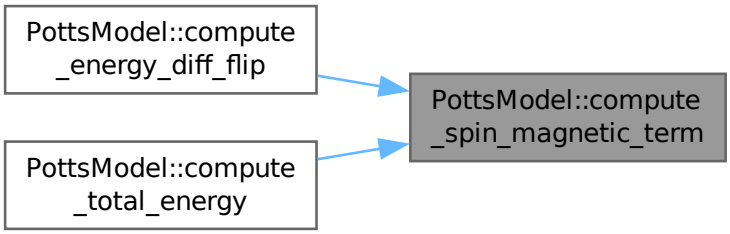
```
double PottsModel::compute_magnetisation ()  [override], [virtual]
```

Implements ModelBase.

compute_spin_magnetic_term() [1/2]

```
int PottsModel::compute_spin_magnetic_term ()  [inline], [override]
```

Here is the caller graph for this function:



compute_spin_magnetic_term() [2/2]

```
double PottsModel::compute_spin_magnetic_term (
            int dim)  [inline], [override], [virtual]
```

computes the spin in the magnetic term of the hamiltonian for a given dimension

Parameters

dim	dimension along which to compute the magnetic term

Returns

double total spin

Implements Model< int >.

compute_spin_neighbours_term() [1/2]

```
int PottsModel::compute_spin_neighbours_term (
            int index)  [override], [virtual]
```

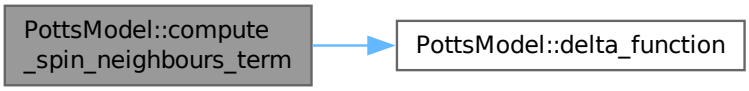
Computes the spin term due to neighbouring spins for a given index.

Returns

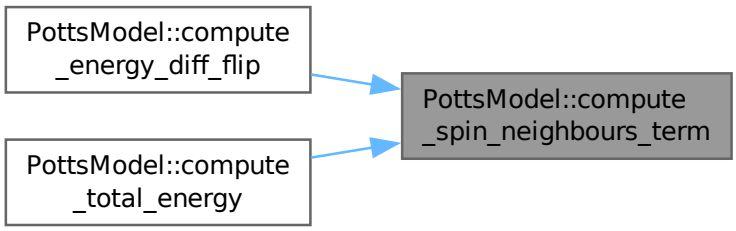
T total spiin

Implements Model< int >.

Here is the call graph for this function:



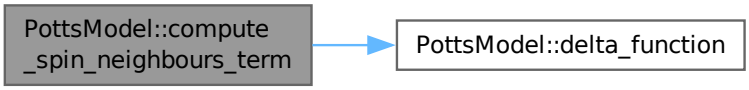
Here is the caller graph for this function:



compute_spin_neighbours_term() [2/2]

```
int PottsModel::compute_spin_neighbours_term (
            ivec indices)  [override]
```

Here is the call graph for this function:

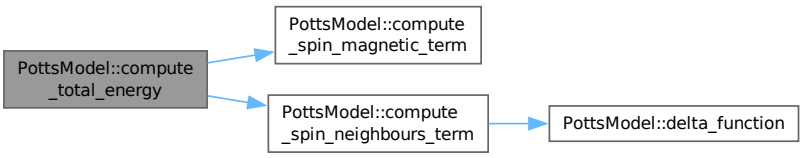


compute_total_energy()

```
double PottsModel::compute_total_energy ()  [override], [virtual]
```

Implements ModelBase.

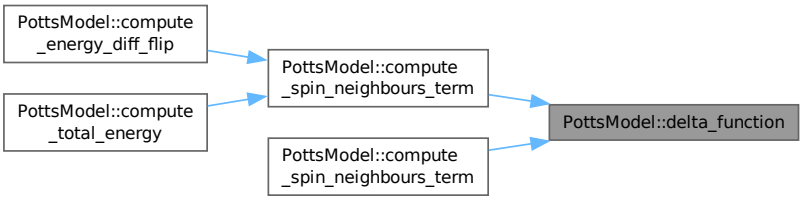
Here is the call graph for this function:



delta_function()

```
int PottsModel::delta_function (
            int a,
            int b)  [private]
```

Here is the caller graph for this function:



8.8.3 Member Data Documentation

H

```
double PottsModel::H  [private]
```

q

```
int PottsModel::q  [private]
```

vec_H

```
dvec PottsModel::vec_H  [private]
```

The documentation for this class was generated from the following files:

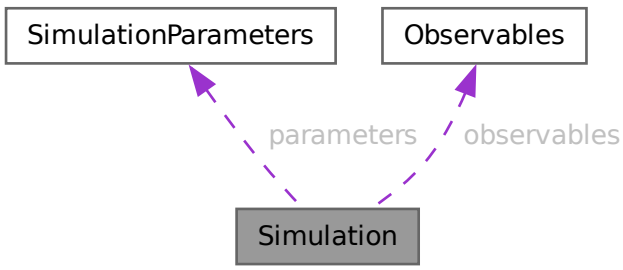
- src/models/potts.h
- src/models/potts.cpp

8.9 Simulation Class Reference

The Simulation class handles the overall simulation process, including parameter parsing, model initialization, running the simulation, and writing outputs.

```
#include <simulation.h>
```

Collaboration diagram for Simulation:



Public Member Functions

- Simulation ()=default

  Construct a new Simulation object.
- void parse_parameters (std::filesystem::path project_folder_path, std::string model_type)

  Parse the parameters from the toml file.
- void initialise_model ()

  Selects the model to be used.
- void initialise_writing ()

  Initialises writing objects.
- void close_writing ()
- void run ()

  It executes the simulation, a full run corresponding to the time steps. Chooses between using renormalisation or not if specified.
- void normal_run ()

  It executes the simulation, a full run corresponding to the time steps.
- void rg_run_2d ()

  It exeucutes the simulation using renormalisation group methods in 2D to store the data for further analysis.
- void rg_run_3d ()

  It exeucutes the simulation using renormalisation group methods in 3D to store the data for further analysis.
- void do_metropolis_step ()

  executes a single Metropolis step
- void do_metropolis_sweep ()

  executes a sweep of the lattice
- void do_metropolis_recording_sweep ()

  Executes a full sweep of Metropolis steps while recording observables.
- void do_cluster_step ()
- void do_cluster_sweep ()
- void write_lattice (int time)

  Writes the lattice in the dataset at a given time step.
- void write_observables_after_loop (hsize_t start)

  writes the observables into the h5 file after the run from the Observables object for the batch, currently it is 2^9 to have easy bitwise
- void write_rg_observables_after_loop (hsize_t start)

  writes the renormalised observables into the h5 file after the run from the Observables object for the batch, currently it is 2^9 to have easy bitwise
- void write_observables_final (hsize_t start, dvec energy, dvec x_magnetisation, dvec y_magnetisation, dvec cluster_size)

  writes the observables into the h5 file for the last batch of steps.
- void write_rg_observables_final (hsize_t start, dvec rg_energy1, dvec rg_energy2, dvec rg_energy3, dvec rg_x_magnetisation1, dvec rg_x_magnetisation2, dvec rg_x_magnetisation3, dvec rg_y_magnetisation1, dvec rg_y_magnetisation2, dvec rg_y_magnetisation3)

  writes the renormalised observables into the h5 file for the last batch of steps.
- void update_observables (int position, ivec &cluster_size)

  Updates the observables stored in the Observables object.
- void update_rg_observables (int position, int b1, int b2, int b3, ivec &neigh_table1, ivec &neigh_table2, ivec &neigh_table3)

  Updates the observables stored in the Observables object corresponding to renormalisation.
- void update_rg_observables_3d (int position, int b1, int b2, ivec &neigh_table1, ivec &neigh_table2)
- double compute_average_cluster_size (ivec &cluster_size)

Private Member Functions

- ivec extract_int_vector_toml (const toml::array &array)

  Extract an integer vector from a toml array.
- dvec extract_double_vector_toml (const toml::array &array)

  Extract a double vector from a toml array.

Private Attributes

- SimulationParameters parameters
- Observables observables
- std::unique_ptr< HighFive::File > file
- int time_step = 0
- int spins_flipped = 0
- ivec cluster_stack
- ivec cluster_size
- std::vector< uint8_t > visited
- std::unique_ptr< ModelBase > model

8.9.1 Detailed Description

The Simulation class handles the overall simulation process, including parameter parsing, model initialization, running the simulation, and writing outputs.

8.9.2 Constructor & Destructor Documentation

Simulation()

```
Simulation::Simulation () [default]
```

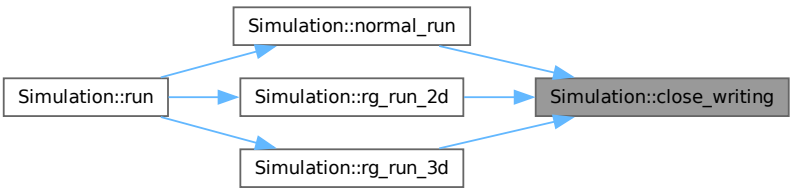
Construct a new Simulation object.

8.9.3 Member Function Documentation

close_writing()

```
void Simulation::close_writing ()
```

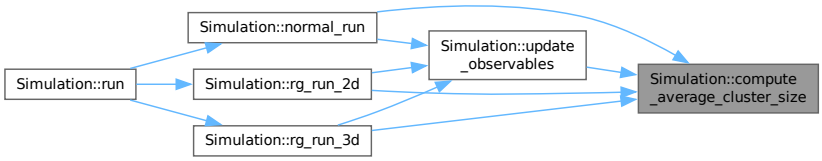
Here is the caller graph for this function:



compute_average_cluster_size()

```
double Simulation::compute_average_cluster_size (
            ivec & cluster_size)
```

Here is the caller graph for this function:

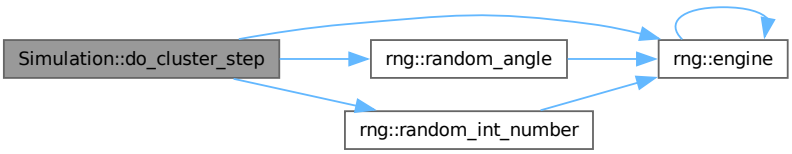


do_cluster_step()

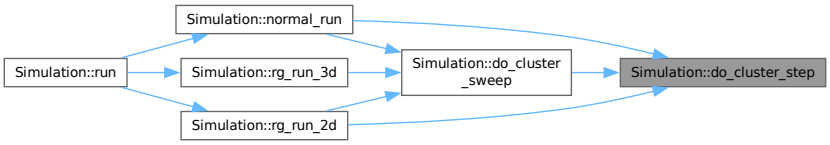
```
void Simulation::do_cluster_step ()
```

Here we need to normalise the MCS steps things (or not) I can do this like this at the moment and leave it.

Total MCS = {\sum (size of each cluster)} / number particles Here is the call graph for this function:



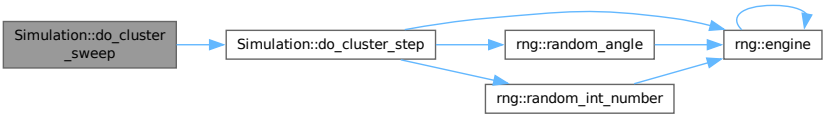
Here is the caller graph for this function:



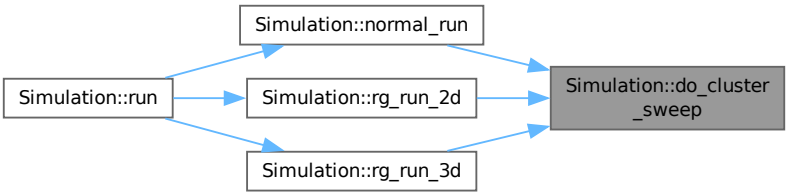
do_cluster_sweep()

```
void Simulation::do_cluster_sweep ()
```

Here is the call graph for this function:



Here is the caller graph for this function:



do_metropolis_recording_sweep()

```
void Simulation::do_metropolis_recording_sweep ()
```

Executes a full sweep of Metropolis steps while recording observables.

TODO: explain algorithm

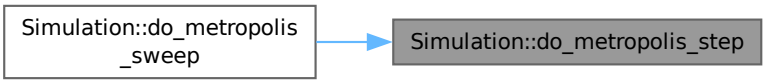
do_metropolis_step()

```
void Simulation::do_metropolis_step ()
```

executes a single Metropolis step

Here is the call graph for this function:

Here is the caller graph for this function:



do_metropolis_sweep()

```
void Simulation::do_metropolis_sweep ()
```

executes a sweep of the lattice

Here time is measured as MCS/site, Monte Carlo Steps per site. So 1 step is N flips for N particles Here is the call graph for this function:

extract_double_vector_toml()

```
dvec Simulation::extract_double_vector_toml (
            const toml::array & array)  [private]
```

Extract a double vector from a toml array.

Parameters

array	array inherited from toml obtained from the file

Returns

dvec data of toml array in double vector

extract_int_vector_toml()

```
ivec Simulation::extract_int_vector_toml (
            const toml::array & array)  [private]
```

Extract an integer vector from a toml array.

Parameters

array	array inherited from toml obtained from the file

Returns

ivec data of toml array in int vector

initialise_model()

```
void Simulation::initialise_model ()
```

Selects the model to be used.

Given a model parsed from the terminal it assigns the ModelBase pointer to the correpsondent model class to execute the simulation

initialise_writing()

```
void Simulation::initialise_writing ()
```

Initialises writing objects.

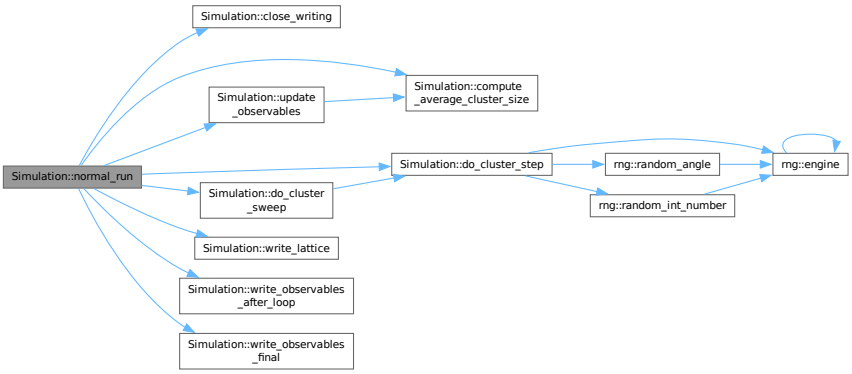
Given the file pointer it creates the h5 file corresponding to that run. It also creates the lattice dataset and dataspace as well as reserving space and defining the chunk size for later writing.

normal_run()

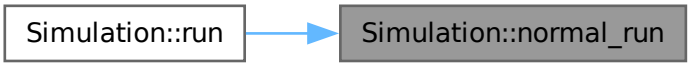
```
void Simulation::normal_run ()
```

It executes the simulation, a full run corresponding to the time steps.

Here is the call graph for this function:



Here is the caller graph for this function:



parse_parameters()

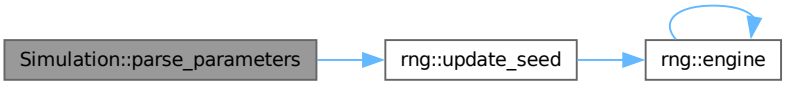
```
void Simulation::parse_parameters (
            std::filesystem::path project_folder_path,
            std::string model_type)
```

Parse the parameters from the toml file.

Parameters

project_folder_path	std::filesystem::path to the project folder
model_type	std::string specifying the model type

Here is the call graph for this function:

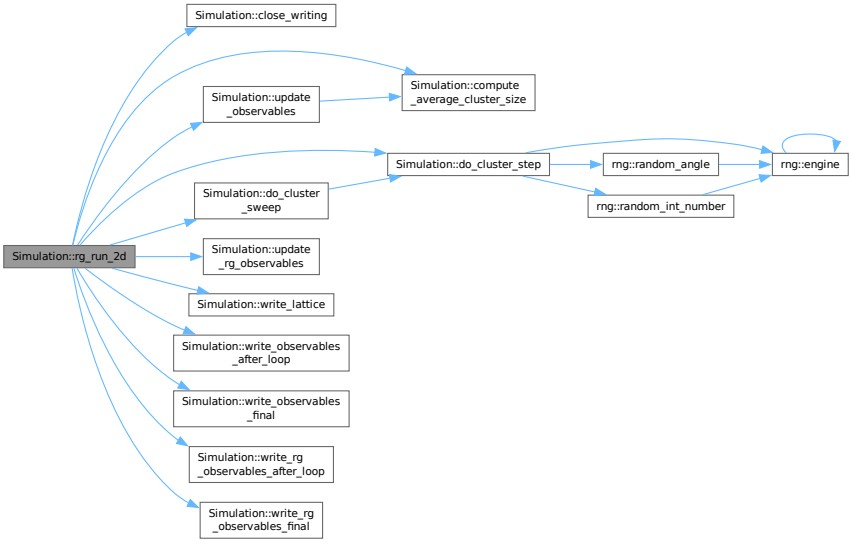


rg_run_2d()

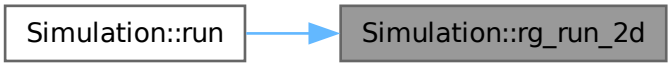
```
void Simulation::rg_run_2d ()
```

It exeucutes the simulation using renormalisation group methods in 2D to store the data for further analysis.

Here is the call graph for this function:



Here is the caller graph for this function:

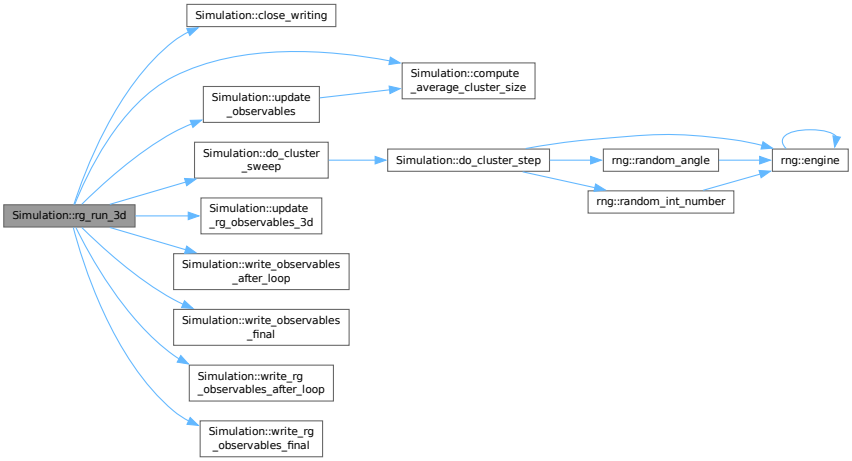


rg_run_3d()

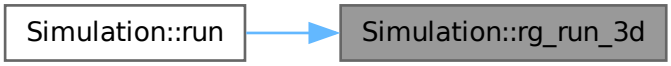
```
void Simulation::rg_run_3d ()
```

It exeucutes the simulation using renormalisation group methods in 3D to store the data for further analysis.

Here is the call graph for this function:



Here is the caller graph for this function:

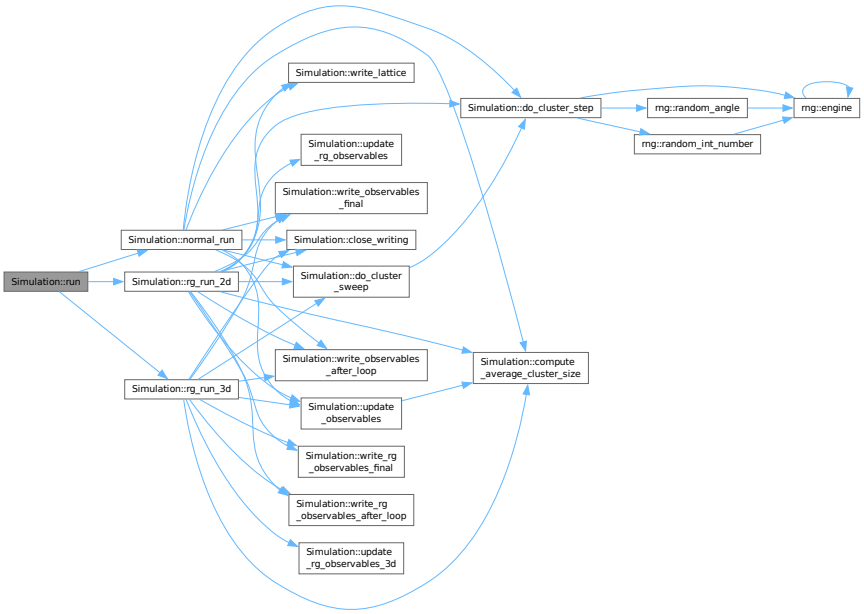


run()

```
void Simulation::run ()
```

It executes the simulation, a full run corresponding to the time steps. Chooses between using renormalisation or not if specified.

Here is the call graph for this function:



update_observables()

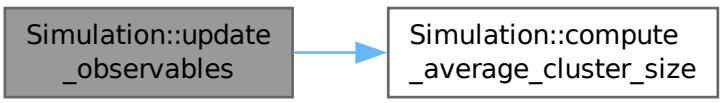
```
void Simulation::update_observables (
            int position,
            ivec & cluster_size)
```

Updates the observables stored in the Observables object.

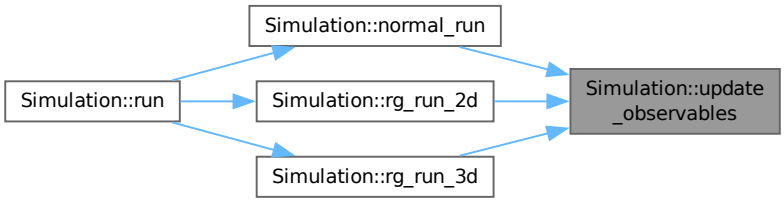
Parameters

position	int gets the position of the array to save observable

Here is the call graph for this function:



Here is the caller graph for this function:



update_rg_observables()

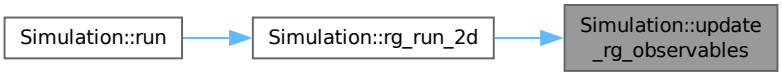
```
void Simulation::update_rg_observables (
            int position,
            int b1,
            int b2,
            int b3,
            ivec & neigh_table1,
            ivec & neigh_table2,
            ivec & neigh_table3)
```

Updates the observables stored in the Observables object corresponding to renormalisation.

Parameters

position	int gets the position of the array to save observable
b1	int first scale factor used
b2	int second scale factor used
neigh_table1	int vector neighbours table for b1
neigh_table2	int vector neighbours table for b2

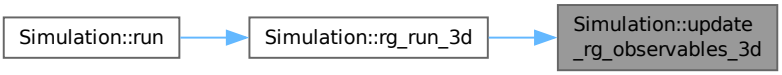
Here is the caller graph for this function:



update_rg_observables_3d()

```
void Simulation::update_rg_observables_3d (
            int position,
            int b1,
            int b2,
            ivec & neigh_table1,
            ivec & neigh_table2)
```

Here is the caller graph for this function:



write_lattice()

```
void Simulation::write_lattice (
            int time)
```

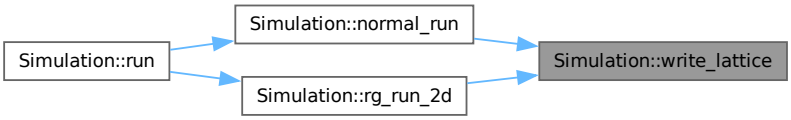
Writes the lattice in the dataset at a given time step.

It resizes the dataset to accomodate the new time step and writes the lattice into the dataset stored in the SimulationParameters struct.

Parameters

time	int time step

Here is the caller graph for this function:



write_observables_after_loop()

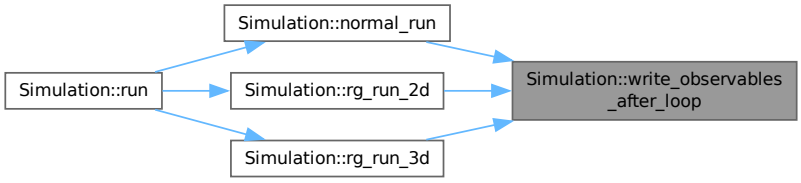
```
void Simulation::write_observables_after_loop (
            hsize_t start)
```

writes the observables into the h5 file after the run from the Observables object for the batch, currently it is 2^9 to have easy bitwise

Parameters

start	int position from which to start writing

Here is the caller graph for this function:



write_observables_final()

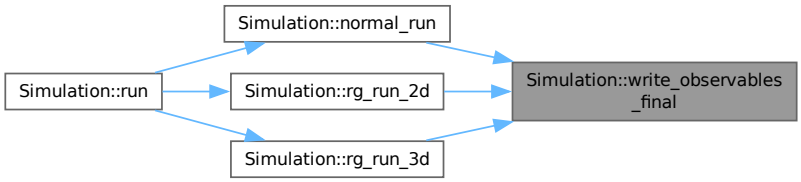
```
void Simulation::write_observables_final (
            hsize_t start,
            dvec energy,
            dvec x_magnetisation,
            dvec y_magnetisation,
            dvec cluster_size)
```

writes the observables into the h5 file for the last batch of steps.

Parameters

start	int position from which to start writing

Here is the caller graph for this function:



write_rg_observables_after_loop()

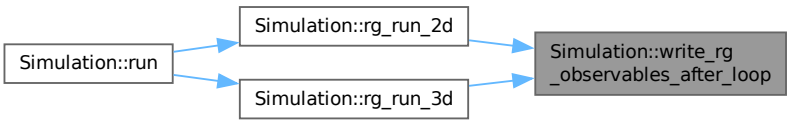
```
void Simulation::write_rg_observables_after_loop (
            hsize_t start)
```

writes the renormalised observables into the h5 file after the run from the Observables object for the batch, currently it is 2^9 to have easy bitwise

Parameters

start	int position from which to start writing

Here is the caller graph for this function:



write_rg_observables_final()

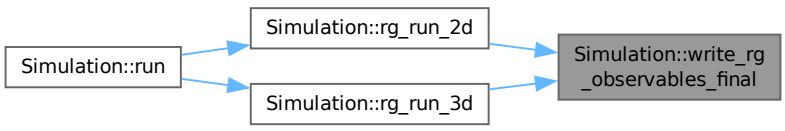
```
void Simulation::write_rg_observables_final (
            hsize_t start,
            dvec rg_energy1,
            dvec rg_energy2,
            dvec rg_energy3,
            dvec rg_x_magnetisation1,
            dvec rg_x_magnetisation2,
            dvec rg_x_magnetisation3,
            dvec rg_y_magnetisation1,
            dvec rg_y_magnetisation2,
            dvec rg_y_magnetisation3)
```

writes the renormalised observables into the h5 file for the last batch of steps.

Parameters

start	int position from which to start writing

Here is the caller graph for this function:



8.9.4 Member Data Documentation

cluster_size

```
ivec Simulation::cluster_size [private]
```

cluster_stack

```
ivec Simulation::cluster_stack [private]
```

file

```
std::unique_ptr<HighFive::File> Simulation::file [private]
```

a unique_ptr element that points to the HDF5 output file

model

```
std::unique_ptr<ModelBase> Simulation::model [private]
```

a unique_ptr element that points to any of the models created with the Model class or its parent ModelBase

observables

```
Observables Simulation::observables [private]
```

an Observables object that stores measurements during the simulation

parameters

```
SimulationParameters Simulation::parameters [private]
```

a SimulationParameters object that stores all the parameters needed for the simulation

spins_flipped

```
int Simulation::spins_flipped = 0 [private]
```

time_step

```
int Simulation::time_step = 0 [private]
```

visited

```
std::vector<uint8_t> Simulation::visited  [private]
```

The documentation for this class was generated from the following files:

- src/simulation/simulation.h
- src/simulation/simulation.cpp

8.10 SimulationParameters Struct Reference

```
#include <simulationParameters.h>
```

Public Attributes

- std::unique_ptr< HighFive::DataSet > lattice_set
- std::unique_ptr< HighFive::DataSet > magnetisation_set
- std::unique_ptr< HighFive::DataSet > energy_set
- std::unique_ptr< HighFive::DataSet > rg_magnetisation_set1
- std::unique_ptr< HighFive::DataSet > rg_magnetisation_set2
- std::unique_ptr< HighFive::DataSet > rg_magnetisation_set3
- std::unique_ptr< HighFive::DataSet > rg_energy_set1
- std::unique_ptr< HighFive::DataSet > rg_energy_set2
- std::unique_ptr< HighFive::DataSet > rg_energy_set3
- std::unique_ptr< HighFive::DataSet > cluster_size_set
- std::string model_type
- std::filesystem::path project_folder_path
- int L
- int dim
- size_t N
- double T
- double beta
- double J
- int potts_q = 1
- size_t total_sweeps
- bool rg_method = false

8.10.1 Member Data Documentation

beta

```
double SimulationParameters::beta
```

cluster_size_set

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::cluster_size_set
```

dim

```
int SimulationParameters::dim
```

energy_set

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::energy_set
```

J

```
double SimulationParameters::J
```

L

```
int SimulationParameters::L
```

lattice_set

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::lattice_set
```

magnetisation_set

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::magnetisation_set
```

model_type

```
std::string SimulationParameters::model_type
```

N

```
size_t SimulationParameters::N
```

potts_q

```
int SimulationParameters::potts_q = 1
```

project_folder_path

```
std::filesystem::path SimulationParameters::project_folder_path
```

rg_energy_set1

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::rg_energy_set1
```

rg_energy_set2

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::rg_energy_set2
```

rg_energy_set3

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::rg_energy_set3
```

rg_magnetisation_set1

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::rg_magnetisation_set1
```

rg_magnetisation_set2

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::rg_magnetisation_set2
```

rg_magnetisation_set3

```
std::unique_ptr<HighFive::DataSet> SimulationParameters::rg_magnetisation_set3
```

rg_method

```
bool SimulationParameters::rg_method = false
```

T

```
double SimulationParameters::T
```

total_sweeps

```
size_t SimulationParameters::total_sweeps
```

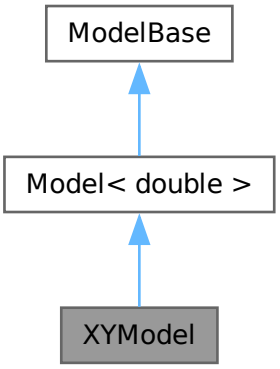
The documentation for this struct was generated from the following file:

- src/simulation/simulationParameters.h

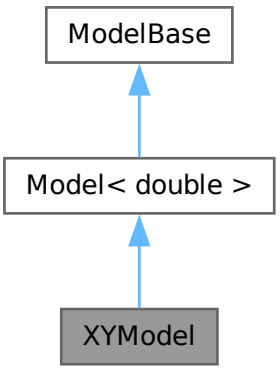
8.11 XYModel Class Reference

```
#include <xy.h>
```

Inheritance diagram for XYModel:



Collaboration diagram for XYModel:



Public Member Functions

- XYModel (double temp, double J, int dim, int L)
- double compute_spin_neighbours_term (int index) override

  Computes the spin term due to neighbouring spins for a given index.
- double compute_spin_neighbours_term (ivec indices) override
- double compute_spin_magnetic_term (int dim) override

computes the spin in the magnetic term of the hamiltonian for a given dimension

- double compute_spin_magnetic_term () override
- double compute_total_energy () override
- double compute_magnetisation () override
- double compute_energy_diff_flip () override
- void cluster_flip_neighbours (int index, double direction, ivec &cluster_stack, int &spins_flipped, std::vector< uint8_t > &visited, int lattice_dim) override
- void change_spin_randomly (ivec indices) override

  Given a site lattice index, it randomly changes the spin at that site.
- void change_spin (int index, double spin) override
- void flip_spin (int index, double angle) override
- void compute_reduced_lattice (int b, size_t N, size_t L) override
- void compute_reduced_lattice_3d (int b, size_t N, size_t L) override
- double compute_rg_spin_magnetic_term (int dim) override
- double compute_rg_energy (int b, size_t N, ivec &neigh_table, int dim) override
- ivec calculate_reduced_neighbours_table (int L, int dim, int b) override

Public Member Functions inherited from Model< double >

- virtual ~Model ()=default

  Destroy the Model object.
- Model (double temp, double J)

  Default Constructor.
- double compute_average_energy (std::vector< double > energies)

  Computes the average energy of the system.
- double compute_heat_capacity (std::vector< double > energies)

  Computes the heat capacity of the system.
- double compute_single_partition_function () override

  Computes the partition function for a single configuration of the system.
- void revert_state_of_system () override

  Given an old configuration it reverts the system to that state.
- void write_lattice (HighFive::DataSet *lattice_set, hsize_t time, hsize_t N) override

  Given a DataSet it writes the current time step lattice configuration to it.

Public Member Functions inherited from ModelBase

- virtual ~ModelBase ()=default

Private Attributes

- dvec vec_H
- double H
- dvec rg_x_spins
- dvec rg_y_spins

Additional Inherited Members

Protected Attributes inherited from Model< double >

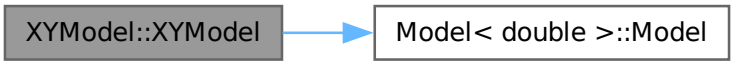
- double beta
- double J
- double temp
- std::unique_ptr< Lattice< double > > lattice_obj
- std::vector< double > old_lattice

8.11.1 Constructor & Destructor Documentation

XYModel()

```
XYModel::XYModel (
            double temp,
            double J,
            int dim,
            int L)  [inline]
```

Here is the call graph for this function:



8.11.2 Member Function Documentation

calculate_reduced_neighbours_table()

```
ivec XYModel::calculate_reduced_neighbours_table (
            int L,
            int dim,
            int b)  [override], [virtual]
```

Reimplemented from Model< double >.

change_spin()

```
void XYModel::change_spin (
            int index,
            double spin)  [override], [virtual]
```

Reimplemented from Model< double >.

change_spin_randomly()

```
void XYModel::change_spin_randomly (
            ivec indices)  [override], [virtual]
```

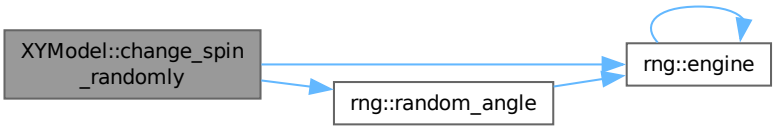
Given a site lattice index, it randomly changes the spin at that site.

Parameters

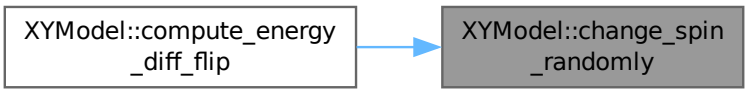
indices	array of indices where the spins are to be changed

Implements Model< double >.

Here is the call graph for this function:



Here is the caller graph for this function:



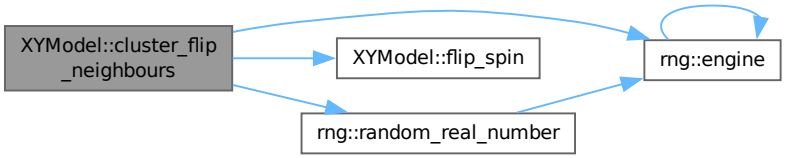
cluster_flip_neighbours()

```
void XYModel::cluster_flip_neighbours (
            int index,
            double direction,
            ivec & cluster_stack,
            int & spins_flipped,
            std::vector< uint8_t > & visited,
            int lattice_dim)  [override], [virtual]
```

) $$ For $\mathbf{r} = (\cos\phi, \sin\phi)$ and $\mathbf{s}_i = (\cos\theta_i, \sin\theta_i)$.

Reimplemented from Model< double >.

Here is the call graph for this function:

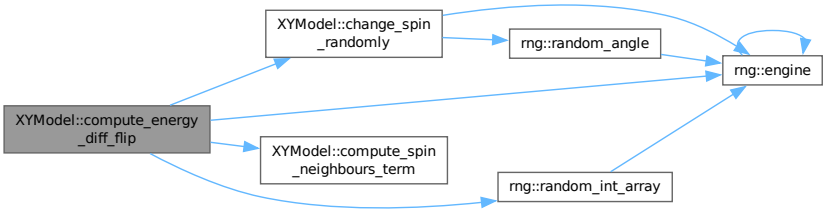


compute_energy_diff_flip()

```
double XYModel::compute_energy_diff_flip () [override], [virtual]
```

Implements ModelBase.

Here is the call graph for this function:

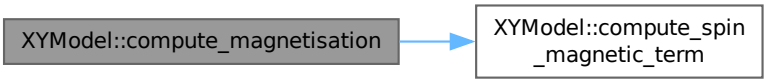


compute_magnetisation()

```
double XYModel::compute_magnetisation () [override], [virtual]
```

Implements ModelBase.

Here is the call graph for this function:



compute_reduced_lattice()

```
void XYModel::compute_reduced_lattice (
            int b,
            size_t N,
            size_t L)  [override], [virtual]
```

Reimplemented from Model< double >.

compute_reduced_lattice_3d()

```
void XYModel::compute_reduced_lattice_3d (
            int b,
            size_t N,
            size_t L)  [override], [virtual]
```

Reimplemented from Model< double >.

compute_rg_energy()

```
double XYModel::compute_rg_energy (
            int b,
            size_t N,
            ivec & neigh_table,
            int dim)  [override], [virtual]
```

Reimplemented from Model< double >.

compute_rg_spin_magnetic_term()

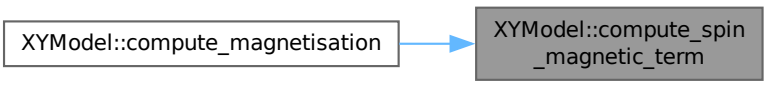
```
double XYModel::compute_rg_spin_magnetic_term (
            int dim)  [override], [virtual]
```

Reimplemented from Model< double >.

compute_spin_magnetic_term() [1/2]

```
double XYModel::compute_spin_magnetic_term ()  [inline], [override]
```

Here is the caller graph for this function:



compute_spin_magnetic_term() [2/2]

```
double XYModel::compute_spin_magnetic_term (
            int dim)  [override], [virtual]
```

computes the spin in the magnetic term of the hamiltonian for a given dimension

Parameters

dim	dimension along which to compute the magnetic term

Returns

double total spin

Implements Model< double >.

compute_spin_neighbours_term() [1/2]

```
double XYModel::compute_spin_neighbours_term (
            int index)  [override], [virtual]
```

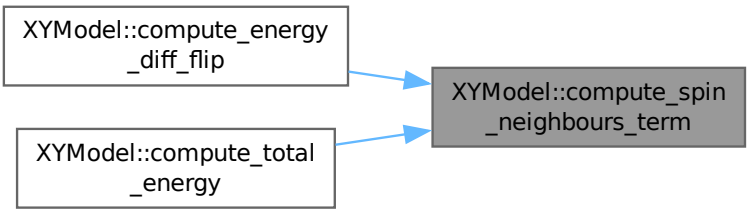
Computes the spin term due to neighbouring spins for a given index.

Returns

T total spiin

Implements Model< double >.

Here is the caller graph for this function:



compute_spin_neighbours_term() [2/2]

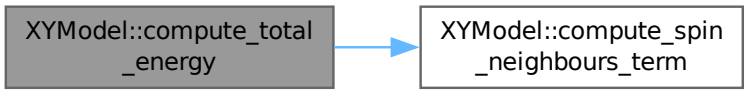
```
double XYModel::compute_spin_neighbours_term (
            ivec indices)  [override]
```

compute_total_energy()

```
double XYModel::compute_total_energy ()  [override], [virtual]
```

Implements ModelBase.

Here is the call graph for this function:

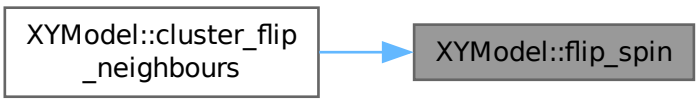


flip_spin()

```
void XYModel::flip_spin (
            int index,
            double angle)  [override], [virtual]
```

Reimplemented from Model< double >.

Here is the caller graph for this function:



8.11.3 Member Data Documentation

H

```
double XYModel::H  [private]
```

rg_x_spins

```
dvec XYModel::rg_x_spins  [private]
```

rg_y_spins

```
dvec XYModel::rg_y_spins  [private]
```

vec_H

```
dvec XYModel::vec_H  [private]
```

The documentation for this class was generated from the following files:

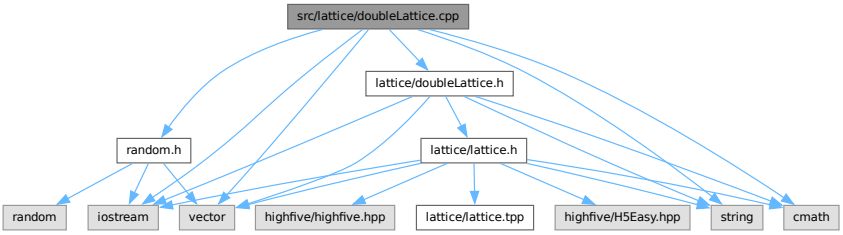
- src/models/xy.h
- src/models/xy.cpp

9 File Documentation

9.1 src/lattice/doubleLattice.cpp File Reference

```
#include "random.h"
#include "lattice/doubleLattice.h"
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
```

Include dependency graph for doubleLattice.cpp:



Typedefs

- using dvec = std::vector<double>

9.1.1 Typedef Documentation

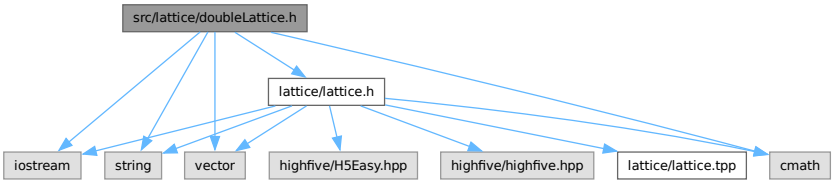
dvec

```
using dvec = std::vector<double>
```

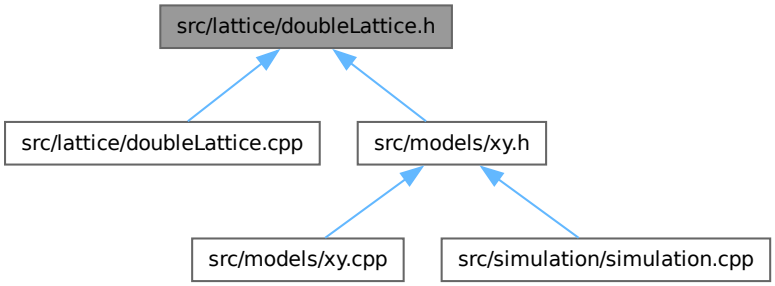
9.2 src/lattice/doubleLattice.h File Reference

```
#include "lattice/lattice.h"
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
```

Include dependency graph for doubleLattice.h:



This graph shows which files directly or indirectly include this file:



Classes

- class DoubleLattice

Typedefs

- using dvec = std::vector<double>
- using ivec = std::vector<int>

9.2.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

ivec

```
using ivec = std::vector<int>
```

9.3 doubleLattice.h

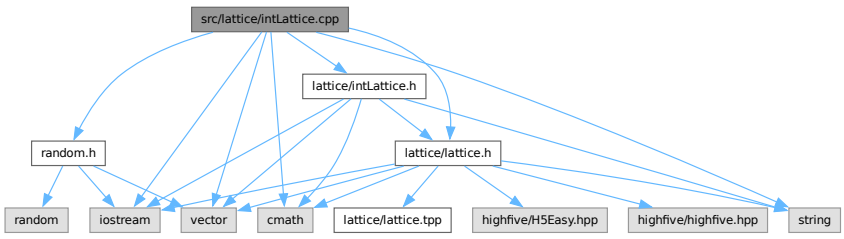
Go to the documentation of this file.

```
#ifndef DOUBLE_LATTICE_H
#define DOUBLE_LATTICE_H
#include "lattice/lattice.h"
#include <iostream>
#include <string>
#include <vector>
#include <cmath>

using dvec = std::vector<double>;
using ivec = std::vector<int>;

class DoubleLattice : public Lattice<double> {

    private:
       dvec generate_lattice(int L, int dim) override;

    public:
        DoubleLattice(const int L, const int dim);

        double& get_lattice_site(ivec indices) override;
        double& get_lattice_site(int index) override;

        dvec get_neighbours_array(ivec indices) override;
        dvec get_neighbours_array(int index) override;

};

#endif
```

9.4 src/lattice/intLattice.cpp File Reference

```
#include "random.h"
#include "lattice/lattice.h"
#include "lattice/intLattice.h"
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
```

Include dependency graph for intLattice.cpp:



Typedefs

- using ivec = std::vector<int>

9.4.1 Typedef Documentation

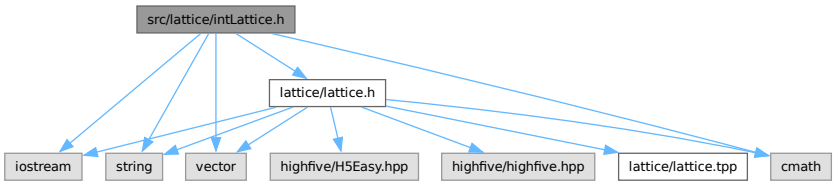
ivec

```
using ivec = std::vector<int>
```

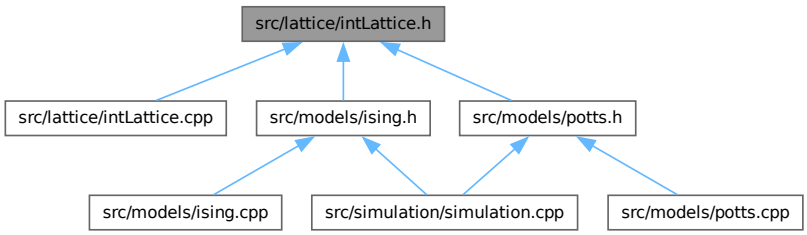
9.5 src/lattice/intLattice.h File Reference

```
#include "lattice/lattice.h"
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
```

Include dependency graph for intLattice.h:



This graph shows which files directly or indirectly include this file:



Classes

- class IntLattice

Typedefs

- using ivec = std::vector<int>

9.5.1 Typedef Documentation

ivec

```
using ivec = std::vector<int>
```

9.6 intLattice.h

Go to the documentation of this file.

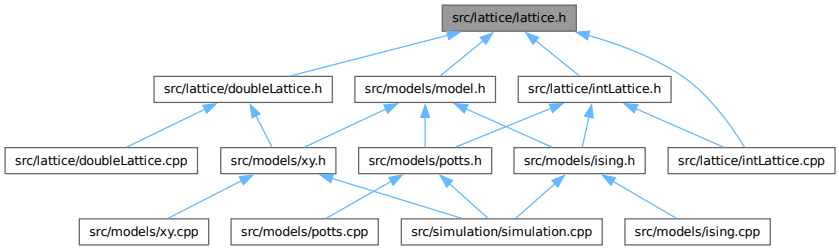
```
00001 #ifndef INT_LATTICE_H
00002 #define INT_LATTICE_H
00003 #include "lattice/lattice.h"
00004 #include <iostream>
00005 #include <string>
00006 #include <vector>
00007 #include <cmath>
00008
00009 using ivec = std::vector<int>;
00010
00011 // Do not forget to make the code more robust and think were to use pointers
00012 class IntLattice : public Lattice<int> {
00013
00014     private:
00015         int potts_q;
00016
00017         // Specifically for Potts' model
00018         ivec generate_lattice(int L, int dim, int q);
00019         ivec generate_lattice(int L, int dim) override;
00020
00021     public:
00022         IntLattice(int L , int dim, int q);
00023         IntLattice(int L , int dim);
00024
00025         int& get_lattice_site(ivec indices) override;
00026         int& get_lattice_site(int index) override;
00027
00028         ivec get_neighbours_array(ivec indices) override;
00029         ivec get_neighbours_array(int index) override;
00030
00031 };
00032
00033 #endif
```

9.7 src/lattice/lattice.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
#include "lattice/lattice.tpp"
```

Include dependency graph for lattice.h:

This graph shows which files directly or indirectly include this file:



Classes

- class Lattice< T >

Typedefs

- using ivec = std::vector<int>
- using dvec = std::vector<double>

9.7.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

ivec

```
using ivec = std::vector<int>
```

9.8 lattice.h

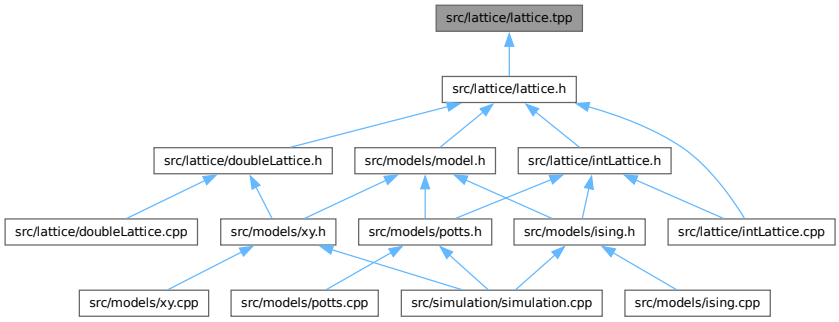
Go to the documentation of this file.

```
#ifndef LATTICE_H
#define LATTICE_H
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>

using ivec = std::vector<int>;
using dvec = std::vector<double>;

// Implement & and stuff asa well as const and maybe virtual functions
template <typename T>
class Lattice {
    protected:
        // Variables or objects
        const int lattice_dim;
```

```
        const int lattice_length;
        const int particle_num;
        ivec neighbours_table;
        std::vector<T> lattice;

        // // Methods
        // Generate the lattices and tables
        virtual std::vector<T> generate_lattice(int L, int dim) = 0;
        ivec calculate_neighbours_table(int L, int dim);

    public:
        // // Probably don't need this
        // // Rule of 5
        // Lattice() = delete;
        // Lattice(const Lattice &) = delete;
        // Lattice &operator=(const Lattice &other) = delete;
        // Lattice(Lattice &&) = delete;
        // Lattice &operator=(Lattice &&) = delete;
        virtual ~Lattice() =default;

        // Constructor
        Lattice(int L, int dim): lattice_dim(dim), lattice_length(L), particle_num(std::pow(L, dim))
    {} // Done

        // Getters
        int get_1d_index(ivec indices);
        //TODO is this redundant??
        int get_lattice_total_length() {return std::pow(lattice_length, lattice_dim);};
        int get_lattice_length() { return lattice_length;}
        int get_lattice_dim() { return lattice_dim;}
        int get_particle_num() { return particle_num;}
        const std::vector<T>& get_lattice() const {return lattice;}
        std::vector<T>& get_lattice(){return lattice;}
        const std::vector<int>& get_neighbours_table() const {return neighbours_table;}
        // // Ver fast way but with another library I think
        // const int* get_neighbours_table_ptr() const {return neighbours_table.data();}

        //Type specific
        virtual T& get_lattice_site(ivec indices) = 0;
        virtual T& get_lattice_site(int index) = 0;

        virtual std::vector<T> get_neighbours_array(ivec indices) = 0;
        virtual std::vector<T> get_neighbours_array(int index) = 0;

};

#include "lattice/lattice.tpp"

#endif
```

9.9 src/lattice/lattice.tpp File Reference

This graph shows which files directly or indirectly include this file:



9.10 lattice.tpp

Go to the documentation of this file.

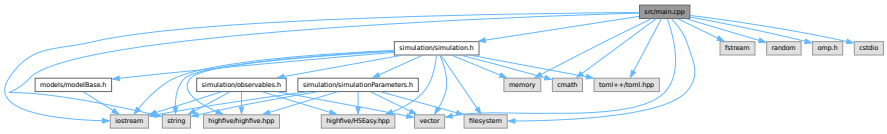
```
#include "lattice/lattice.h"
#include "random.h"
#include <cassert>
#include <cmath>
#include <iostream>
#include <string>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>

using ivec = std::vector<int>;
using dvec = std::vector<double>;

// (i,j) -> i*L + j;
// (i1, i2, ..., in) -> i1 * L^n + i2 * L^{n-1} + ... + i(n-1) * L + in
template<class T>
int Lattice<T>::get_1d_index(ivec indices) {
    assert((lattice_dim == indices.size()) &&
           "Indices provided need to be same size as lattice");

    int index = 0;

    for (int i = 0; i < lattice_dim; i++) {
        assert((indices[i] < lattice_length) && "Indices must be in the lattice");
        index += indices[i] * std::pow(lattice_length, lattice_dim - i - 1);
    }

    return index;
}


template<class T>
ivec Lattice<T>::calculate_neighbours_table(int L, int dim) {
    ivec neighbours_array(std::pow(L, dim) * 2 * dim);
    if (dim == 2) {
        for (int i = 0; i < L; i++) {
            for (int j = 0; j < L; j++) {

                int index = i * L + j;

                // i
                int i_up = (i + 1 == L) ? 0 : i + 1;
                int i_down = (i - 1 == -1) ? L - 1 : i - 1;
                // j
                int j_up = (j + 1 == L) ? 0 : j + 1;
                int j_down = (j - 1 == -1) ? L - 1 : j - 1;

                // Up
                neighbours_array[4 * index + 0] = i_up * L + j;
                // Down
                neighbours_array[4 * index + 1] = i_down * L + j;
                // Right
                neighbours_array[4 * index + 2] = i* L + j_up;
                // Left
                neighbours_array[4 * index + 3] = i * L + j_down;
            }
        }

    } else if (dim == 3) {
        for (int i = 0; i < L; i++) {
            for (int j = 0; j < L; j++) {
                for (int k = 0; k < L; k++) {

                    int index = i * L * L + j * L + k;

                    // i
                    int i_up = (i + 1 == L) ? 0 : i + 1;
                    int i_down = (i - 1 == -1) ? L - 1 : i - 1;
                    // j
                    int j_up = (j + 1 == L) ? 0 : j + 1;
                    int j_down = (j - 1 == -1) ? L - 1 : j - 1;
                    // k
                    int k_up = (k + 1 == L) ? 0 : k + 1;
                    int k_down = (k - 1 == -1) ? L - 1 : k - 1;

                    // i
                    neighbours_array[6 * index + 0] = i_up * L * L + j * L + k;
                    neighbours_array[6 * index + 1] = i_down * L * L + j * L + k;
                    // j
                    neighbours_array[6 * index + 2] = i * L * L + j_up * L + k;
                    neighbours_array[6 * index + 3] = i * L * L + j_down * L + k;
                    // k
                    neighbours_array[6 * index + 4] = i * L * L + j * L + k_up;
                    neighbours_array[6 * index + 5] = i * L * L + j * L + k_down;
                }
            }
        }

```

```
    } else {
        // TODO Look at obsidian to know how to further implement it
        std::cout « "Further dimensions not implemented yet" « "\n";
        exit(2);
    }
    return neighbours_array;
}
```

9.11 src/main.cpp File Reference

```
#include "simulation/simulation.h"
#include <cmath>
#include <filesystem>
#include <fstream>
#include <iostream>
#include <random>
#include <string>
#include <vector>
#include <memory>
#include <toml++/toml.hpp>
#include <omp.h>
#include <cstdio>
```

Include dependency graph for main.cpp:



Functions

- int main (int argc, char *argv[ ])

  Main function to run the simulation.

9.11.1 Function Documentation

main()

```
int main (
            int argc,
            char * argv[ ])
```

Main function to run the simulation.

Parameters

argc	Number of arguments
argv	Array of arguments

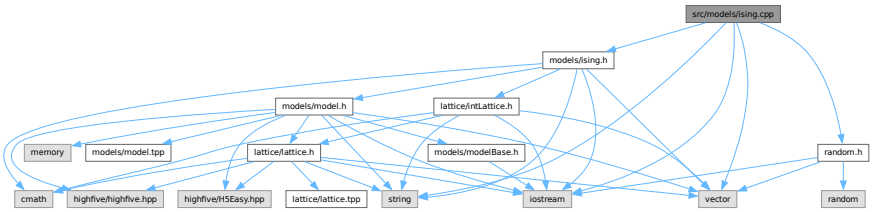
Returns

int Exit code

9.12 src/models/ising.cpp File Reference

```
#include "models/ising.h"
#include "random.h"
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for ising.cpp:



Typedefs

- using ivec = std::vector<int>

9.12.1 Typedef Documentation

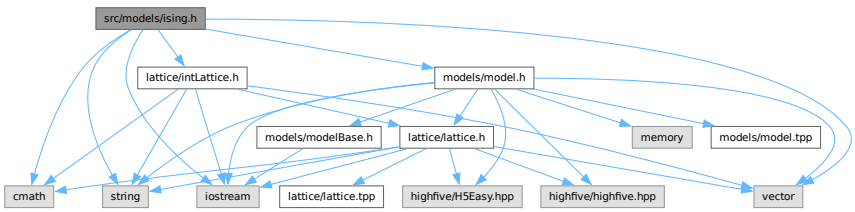
ivec

```
using ivec = std::vector<int>
```

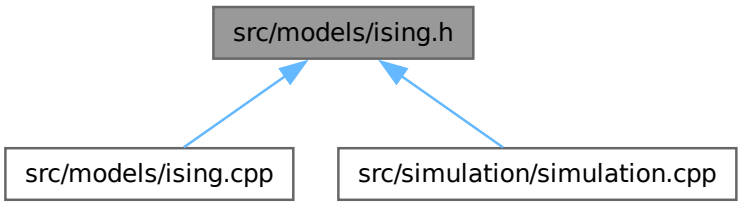
9.13 src/models/ising.h File Reference

```
#include "lattice/intLattice.h"
#include "models/model.h"
#include <cmath>
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for ising.h:



This graph shows which files directly or indirectly include this file:



Classes

- class IsingModel

Typedefs

- using ivec = std::vector<int>

9.13.1 Typedef Documentation

ivec

```
using ivec = std::vector<int>
```

9.14 ising.h

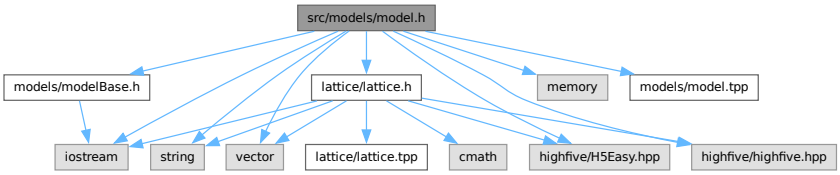
Go to the documentation of this file.

```
#ifndef ISINGMODEL_H
#define ISINGMODEL_H
#include "lattice/intLattice.h"
#include "models/model.h"
#include <cmath>
#include <iostream>
#include <string>
#include <vector>

using ivec = std::vector<int>;

class IsingModel : public Model<int> {
  private:
    // double H;

  public:
    // IsingModel(double temp, double J, double H, const int dim, const int L)
    //     : Model<int>(temp, J), H(H) {
    IsingModel(double temp, double J, const int dim, const int L)
        : Model<int>(temp, J) {
        this->lattice_obj = std::make_unique<IntLattice>(L,dim);
    }

    int compute_spin_neighbours_term(int index) override;
    int compute_spin_neighbours_term(ivec indices) override;
    int compute_spin_magnetic_term() override;
    double compute_spin_magnetic_term(int dim) override {return 0.;}
    double compute_total_energy() override;
    double compute_magnetisation() override;
    void change_spin_randomly(ivec indices) override;
    double compute_energy_diff_flip() override;
};


#endif
```

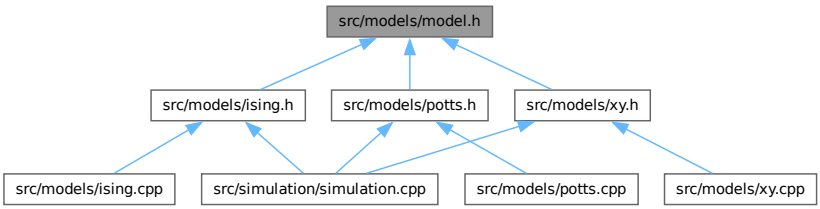
9.15 src/models/model.h File Reference

```
#include "lattice/lattice.h"
#include "models/modelBase.h"
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
#include "models/model.tpp"
```

Include dependency graph for model.h:



This graph shows which files directly or indirectly include this file:



Classes

- class Model< T >

  Base clase for different models.

Typedefs

- using ivec = std::vector<int>

9.15.1 Typedef Documentation

ivec

```
using ivec = std::vector<int>
```

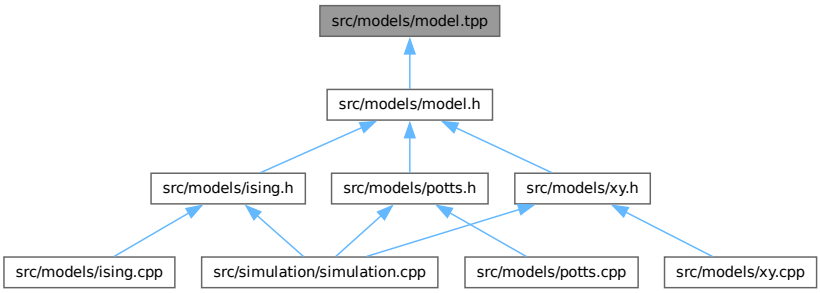
9.16 model.h

Go to the documentation of this file.

```
#ifndef MODEL_H
#define MODEL_H
#include "lattice/lattice.h"
#include "models/modelBase.h"
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>


using ivec = std::vector<int>;

template <typename T>
class Model: public ModelBase{
    protected:
        double beta,J, temp;
        std::unique_ptr<Lattice<T>> lattice_obj;
        std::vector<T> old_lattice;


    public:
        //TODO manage the virutal functions and destructors,
        // copies, etc...

        virtual ~Model() =default;

        Model(double temp, double J)
            : ModelBase(),beta(1/temp), J(J), temp(temp){}

        // Class specific methods
        double compute_average_energy(std::vector<T> energies);

        double compute_heat_capacity(std::vector<T> energies);


        double compute_single_partition_function() override;


        void revert_state_of_system() override {lattice_obj->get_lattice()=old_lattice;}


        void write_lattice(HighFive::DataSet* lattice_set, hsize_t time, hsize_t N) override;

        // Virtual functions
        virtual T compute_spin_neighbours_term(int index) = 0;

        virtual T compute_spin_neighbours_term(ivec indices) = 0;

        virtual double compute_spin_magnetic_term(int dim) = 0;

        virtual T compute_spin_magnetic_term() = 0;

        virtual void change_spin_randomly(ivec indices) = 0;
    // TODO implement this
        void change_spin(int index, double spin) override {}
        void flip_spin(int index, double angle) override {}
        void cluster_flip_neighbours(int index, double direction, ivec& cluster_stack, int&
spins_flipped, std::vector<uint8_t> & visited, int lattice_dim) override{}
        void compute_reduced_lattice(int b, size_t N, size_t L) override{}
        void compute_reduced_lattice_3d(int b, size_t N, size_t L) override{}
        double compute_rg_spin_magnetic_term(int dim) override{return 1.;}
        double compute_rg_energy(int b, size_t N, ivec & neigh_table, int dim) override{return 0.;}
        ivec calculate_reduced_neighbours_table(int L, int dim, int b) override{ivec a{0};return a;}

};

#include "models/model.tpp"

#endif
```

9.17 src/models/model.tpp File Reference

This graph shows which files directly or indirectly include this file:



9.18 model.tpp

Go to the documentation of this file.

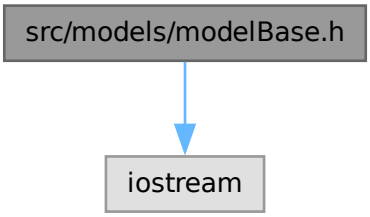
```
#include "lattice/lattice.h"
#include "models/model.h"

#include <cassert>
#include <cmath>
#include <iostream>
#include <string>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>

using ivec = std::vector<int>;
using dvec = std::vector<double>;

// e^-\beta H
template <typename T>
double Model<T>::compute_single_partition_function() {
    T total_energy = compute_total_energy();

    double z = exp(- beta * total_energy);

    return z;
}

template <typename T>
double Model<T>::compute_average_energy(std::vector<T> energies) {
    double avg_e = 0;
    for (auto & i: energies) {
        avg_e += i;
    }

    return avg_e / energies.size();

}

template <typename T>
double Model<T>::compute_heat_capacity(std::vector<T> energies) {
    double c_v = 0;
    double avg_e = compute_average_energy(energies);

    for (auto & i: energies) {
        i *= i;
    }

    double avg_e2 = compute_average_energy(energies);

    c_v = avg_e2 - avg_e * avg_e;

    return c_v;
}

template <typename T>
```

```
00052 void Model<T>::write_lattice(HighFive::DataSet* lattice_set, hsize_t time, hsize_t N) {
00053     const std::vector<T>& lattice = lattice_obj->get_lattice();
00054     lattice_set->select({time, 0},{1,N}).write_raw(lattice.data(), HighFive::AtomicType<double>{});
00055 }
```

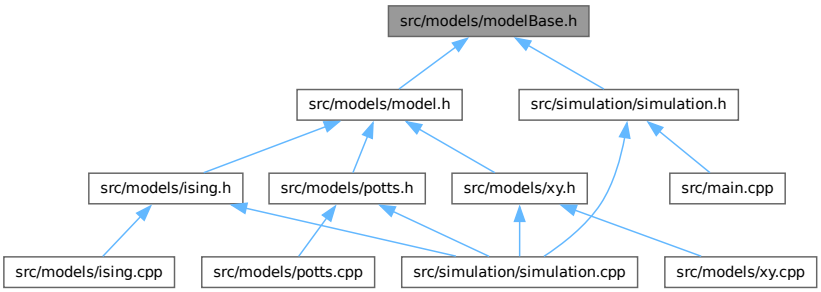
9.19 src/models/modelBase.h File Reference

`#include <iostream>`

Include dependency graph for modelBase.h:



This graph shows which files directly or indirectly include this file:



Classes

- class ModelBase

9.20 modelBase.h

Go to the documentation of this file.

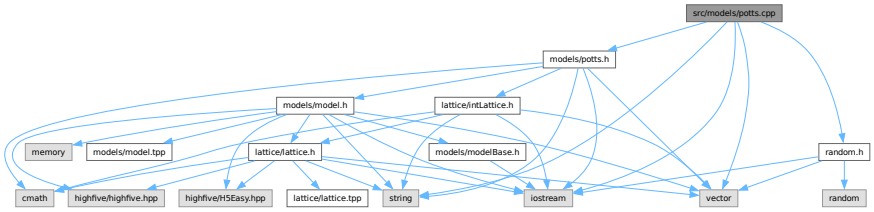
```
00001 #ifndef MODELBASE_H
00002 #define MODELBASE_H
00003 #include <iostream>
00004
00005 class ModelBase {
00006     public:
```

```
        virtual ~ModelBase() = default;
        virtual double compute_single_partition_function() = 0;
        virtual void revert_state_of_system() = 0;
        virtual double compute_total_energy() = 0;
        virtual double compute_magnetisation() = 0;
        virtual double compute_energy_diff_flip() = 0;
        virtual double compute_spin_magnetic_term(int dim) = 0;
        virtual void change_spin(int random_index, double spin) = 0;
        virtual void flip_spin(int index, double angle) = 0;
        virtual void cluster_flip_neighbours(int index, double direction, ivec& cluster_stack, int&
    spins_flipped, std::vector<uint8_t> & visited, int lattice_dim) = 0;
        virtual void write_lattice(HighFive::DataSet * lattice_set, hsize_t time, hsize_t N) = 0;
        virtual void compute_reduced_lattice(int b, size_t N, size_t L) = 0;
        virtual void compute_reduced_lattice_3d(int b, size_t N, size_t L) = 0;
        virtual double compute_rg_spin_magnetic_term(int dim) = 0;
        virtual double compute_rg_energy(int b, size_t N, ivec & neigh_table, int dim) = 0;
        virtual ivec calculate_reduced_neighbours_table(int L, int dim, int b) = 0;
};

#endif
```

9.21 src/models/potts.cpp File Reference

```
#include "models/potts.h"
#include "random.h"
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for potts.cpp:



Typedefs

- using ivec = std::vector<int>

9.21.1 Typedef Documentation

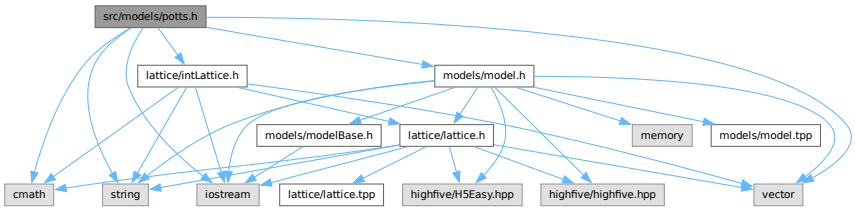
ivec

```
using ivec = std::vector<int>
```

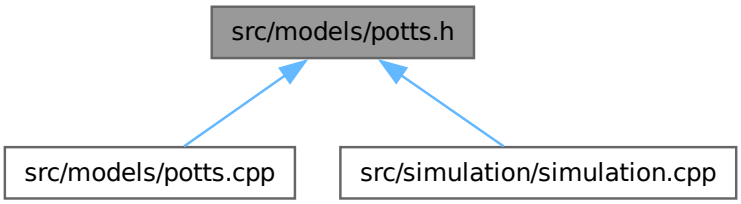
9.22 src/models/potts.h File Reference

```
#include "lattice/intLattice.h"
#include "models/model.h"
#include <cmath>
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for potts.h:



This graph shows which files directly or indirectly include this file:



Classes

- class PottsModel

Typedefs

- using ivec = std::vector<int>
- using dvec = std::vector<double>

9.22.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

ivec

```
using ivec = std::vector<int>
```

9.23 potts.h

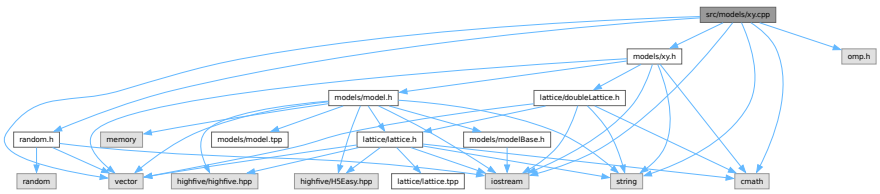
Go to the documentation of this file.

```
#ifndef POTTSMODEL_H
#define POTTSMODEL_H
#include "lattice/intLattice.h"
#include "models/model.h"
#include <cmath>
#include <iostream>
#include <string>
#include <vector>

using ivec = std::vector<int>;
using dvec = std::vector<double>;

class PottsModel : public Model<int> {
  private:
    dvec vec_H;
    double H;
    int q;

    int delta_function(int a, int b);

  public:
    // PottsModel(double temp, double J, dvec vec_H, int H, int dim, int L, int q)
    //     : Model<int>(temp, J), H(H),vec_H(vec_H), q(q) {
    PottsModel(double temp, double J, int dim, int L, int q)
        : Model<int>(temp, J), q(q) {
        this->lattice_obj = std::make_unique<IntLattice>(L,dim,q);
    }

        int compute_spin_neighbours_term(int index) override;
        int compute_spin_neighbours_term(ivec indices) override;
        // int compute_spin_magnetic_term(int dim) override;
        double compute_spin_magnetic_term(int dim) override{return 0.;};
        int compute_spin_magnetic_term() override{return 0;};
        double compute_total_energy() override;
        double compute_magnetisation() override;
        void change_spin_randomly(ivec indices) override;
        double compute_energy_diff_flip() override;
};

#endif
```

9.24 src/models/xy.cpp File Reference

```
#include "models/xy.h"
#include "random.h"
#include <iostream>
#include <string>
#include <omp.h>
#include <vector>
#include <cmath>
```

Include dependency graph for xy.cpp:



Typedefs

- using ivec = std::vector<int>
- using dvec = std::vector<double>

9.24.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

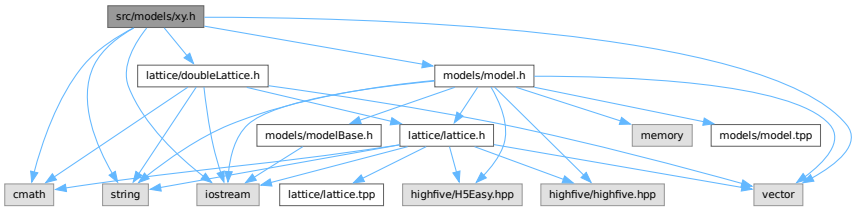
ivec

```
using ivec = std::vector<int>
```

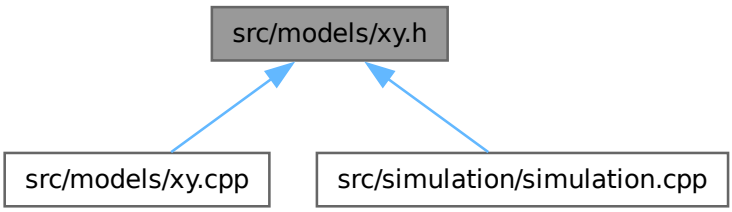
9.25 src/models/xy.h File Reference

```
#include "lattice/doubleLattice.h"
#include "models/model.h"
#include <cmath>
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for xy.h:



This graph shows which files directly or indirectly include this file:



Classes

- class XYModel

Typedefs

- using ivec = std::vector<int>
- using dvec = std::vector<double>

9.25.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

ivec

```
using ivec = std::vector<int>
```

9.26 xy.h

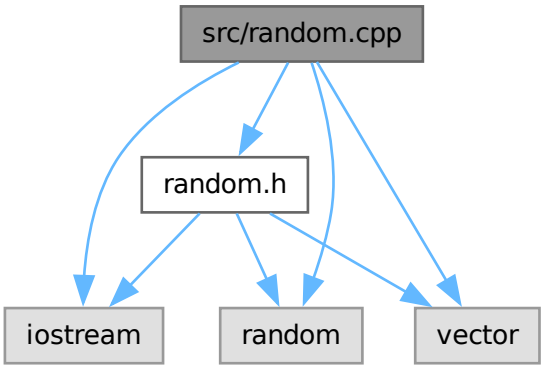
Go to the documentation of this file.

```
#ifndef XYMODEL_H
#define XYMODEL_H
#include "lattice/doubleLattice.h"
#include "models/model.h"
#include <cmath>
#include <iostream>
#include <string>
#include <vector>

using ivec = std::vector<int>;
using dvec = std::vector<double>;

class XYModel : public Model<double> {
  private:
    dvec vec_H;
    double H;
    dvec rg_x_spins;
    dvec rg_y_spins;

  public:
    // XYModel(double temp, double J, dvec vec_H, int H, int dim, int L)
        // : Model<double>(temp, J), H(H),vec_H(vec_H) {
    XYModel(double temp, double J, int dim, int L)
        : Model<double>(temp, J) {
        this->lattice_obj = std::make_unique<DoubleLattice>(L,dim);
    }
        double compute_spin_neighbours_term(int index) override;
        double compute_spin_neighbours_term(ivec indices) override;
        double compute_spin_magnetic_term(int dim) override;
        double compute_spin_magnetic_term() override {return 0;};
        double compute_total_energy() override;
        double compute_magnetisation() override;
        double compute_energy_diff_flip() override;
        void cluster_flip_neighbours(int index, double direction, ivec& cluster_stack, int&
spins_flipped, std::vector<uint8_t> & visited, int lattice_dim) override;
        void change_spin_randomly(ivec indices) override;
        void change_spin(int index, double spin) override;
        void flip_spin(int index, double angle) override;
        void compute_reduced_lattice(int b, size_t N, size_t L) override;
        void compute_reduced_lattice_3d(int b, size_t N, size_t L) override;
        double compute_rg_spin_magnetic_term(int dim) override;
        double compute_rg_energy(int b, size_t N, ivec & neigh_table, int dim) override;
        ivec calculate_reduced_neighbours_table(int L, int dim, int b) override;
};

#endif
```

9.27 src/random.cpp File Reference

```
#include "random.h"
#include <random>
#include <iostream>
#include <vector>
```

Include dependency graph for random.cpp:



Variables

- const double PI = 3.14159265358979323846 3

9.27.1 Variable Documentation

PI

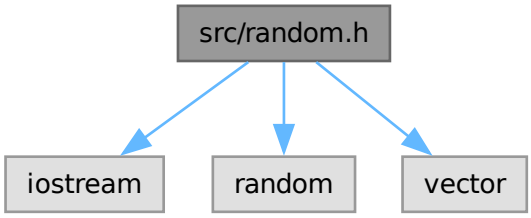
```
const double PI = 3.141592653589793238463
```

9.28 src/random.h File Reference

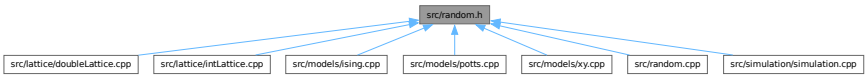
```
#include <iostream>
#include <random>
```

`#include <vector>`

Include dependency graph for random.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace rng

Functions

- std::mt19937 rng::engine (0)
- void rng::update_seed (int new_seed)
- int rng::random_int_number (std::mt19937 &engine, int start=0, int end=1)
- double rng::random_angle (std::mt19937 &engine)
- double rng::random_real_number (std::mt19937 &engine)
- std::vector< int > rng::random_int_array (std::mt19937 &engine, int length, int start=0, int end=1)
- std::vector< double > rng::random_angle_array (std::mt19937 &engine, int length)

9.29 random.h

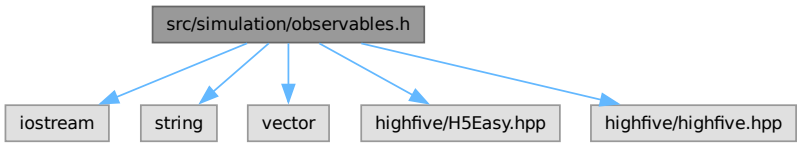
Go to the documentation of this file.

```
#ifndef RANDOM_H
#define RANDOM_H
#include <iostream>
#include <random>
#include <vector>

namespace rng {
    // Be careful with seed!!!
    inline std::mt19937 engine(0);
    void update_seed(int new_seed);
    int random_int_number(std::mt19937 &engine, int start = 0, int end = 1);
    double random_angle(std::mt19937 &engine);
    double random_real_number(std::mt19937 &engine);
    std::vector<int> random_int_array(std::mt19937 &engine, int length,
                                      int start = 0, int end = 1);
    std::vector<double> random_angle_array(std::mt19937 &engine, int length);
} // namespace rng

#endif
```

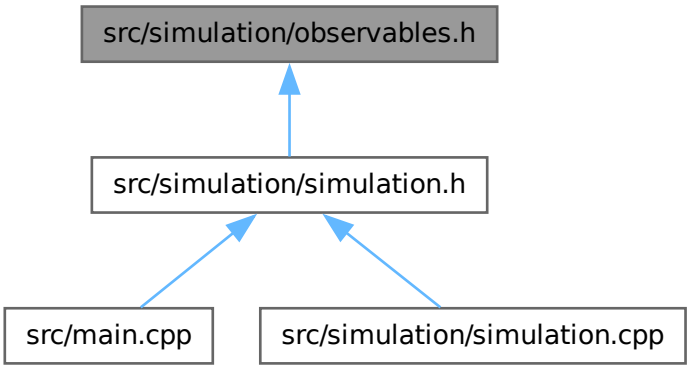
9.30 src/simulation/observables.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
```

Include dependency graph for observables.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct Observables

Typedefs

- using ivec = std::vector<int>
- using dvec = std::vector<double>

9.30.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

ivec

```
using ivec = std::vector<int>
```

9.31 observables.h

Go to the documentation of this file.

```
#ifndef OBSERVABLES_H
#define OBSERVABLES_H
#include <iostream>
#include <string>
#include <vector>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>

using ivec = std::vector<int>;
using dvec = std::vector<double>;

//*********************
// magnetisation
// energy
// susceptibility
// specific heat
// correlation length
// correlation function


struct Observables {
    dvec energy_array;
    dvec x_magnetisation;
    dvec y_magnetisation;
    double susceptibility;
    double correlation_length;
    dvec correlation_function_array;
    dvec average_cluster_size;
    // renormalised quantities
    dvec rg_x_magnetisation1;
    dvec rg_y_magnetisation1;
    dvec rg_x_magnetisation2;
    dvec rg_y_magnetisation2;
    dvec rg_x_magnetisation3;
    dvec rg_y_magnetisation3;
    dvec rg_energy1;
    dvec rg_energy2;
    dvec rg_energy3;
};


#endif
```

9.32 src/simulation/simulation.cpp File Reference

```
#include "simulation/simulation.h"
#include "simulation/simulationParameters.h"
#include "models/ising.h"
#include "models/potts.h"
#include "models/xy.h"
#include "random.h"
#include <iostream>
```

```
#include <filesystem>
#include <vector>
#include <string>
#include <cmath>
#include <omp.h>
#include <chrono>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
#include <toml++/toml.hpp>
```

Include dependency graph for simulation.cpp:

Typedefs

- using ivec = std::vector<int>
- using dvec = std::vector<double>

9.32.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

ivec

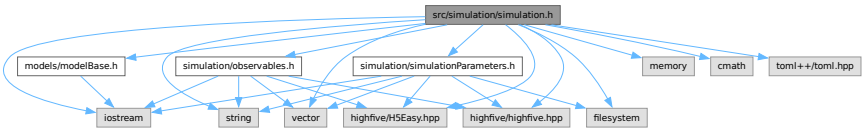
```
using ivec = std::vector<int>
```

9.33 src/simulation/simulation.h File Reference

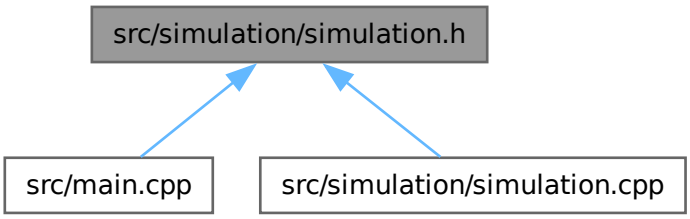
```
#include "simulation/simulationParameters.h"
#include "simulation/observables.h"
#include "models/modelBase.h"
#include <iostream>
#include <vector>
#include <filesystem>
#include <memory>
#include <string>
#include <cmath>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
```

```
#include <toml++/toml.hpp>
```

Include dependency graph for simulation.h:



This graph shows which files directly or indirectly include this file:



Classes

- class Simulation

  The Simulation class handles the overall simulation process, including parameter parsing, model initialization, running the simulation, and writing outputs.

Typedefs

- using ivec = std::vector<int>
- using dvec = std::vector<double>

9.33.1 Typedef Documentation

dvec

```
using dvec = std::vector<double>
```

ivec

```
using ivec = std::vector<int>
```

9.34 simulation.h

Go to the documentation of this file.

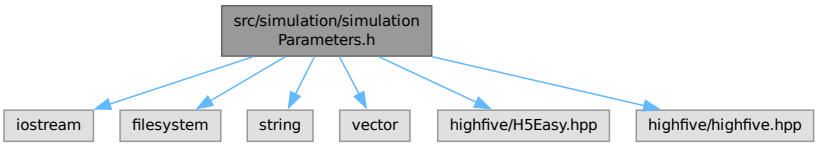
```
#ifndef SIMULATION_H
#define SIMULATION_H
#include "simulation/simulationParameters.h"
#include "simulation/observables.h"
#include "models/modelBase.h"
#include <iostream>
#include <vector>
#include <filesystem>
#include <memory>
#include <string>
#include <cmath>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
#include <toml++/toml.hpp>

using ivec = std::vector<int>;
using dvec = std::vector<double>;

class Simulation {
    private:
        SimulationParameters parameters;
        Observables observables;
        std::unique_ptr<HighFive::File> file;
        int time_step = 0;
        int spins_flipped = 0;
        ivec cluster_stack;
        ivec cluster_size;
        std::vector<uint8_t> visited;
        std::unique_ptr<ModelBase> model;

        ivec extract_int_vector_toml(const toml::array& array);

        dvec extract_double_vector_toml(const toml::array& array);

    public:
        Simulation() = default;

        void parse_parameters(std::filesystem::path project_folder_path, std::string model_type);

        void initialise_model();

        void initialise_writing();

        /* @brief Cleans writing objects
         *
         */
        void close_writing();

        void run();

        void normal_run();

        void rg_run_2d();
        void rg_run_3d();

        void do_metropolis_step();

        void do_metropolis_sweep();

        void do_metropolis_recording_sweep();

        //TODO
        void do_cluster_step();
        void do_cluster_sweep();

        void write_lattice(int time);

        void write_observables_after_loop(hsize_t start);

        void write_rg_observables_after_loop(hsize_t start);

        void write_observables_final(hsize_t start, dvec energy, dvec x_magnetisation,dvec y_magnetisation, dvec cluster_size);

        void write_rg_observables_final(hsize_t start, dvec rg_energy1, dvec rg_energy2, dvec rg_energy3, dvec rg_x_magnetisation1,dvec rg_x_magnetisation2,dvec rg_x_magnetisation3, dvec rg_y_magnetisation1,dvec rg_y_magnetisation2, dvec rg_y_magnetisation3);

        void update_observables(int position, ivec & cluster_size);

        void update_rg_observables(int position, int b1, int b2,int b3, ivec & neigh_table1, ivec & neigh_table2, ivec & neigh_table3);
```

```
        void update_rg_observables_3d(int position, int b1, int b2, ivec & neigh_table1, ivec & neigh_table2);

        double compute_average_cluster_size(ivec & cluster_size);
};

#endif
```

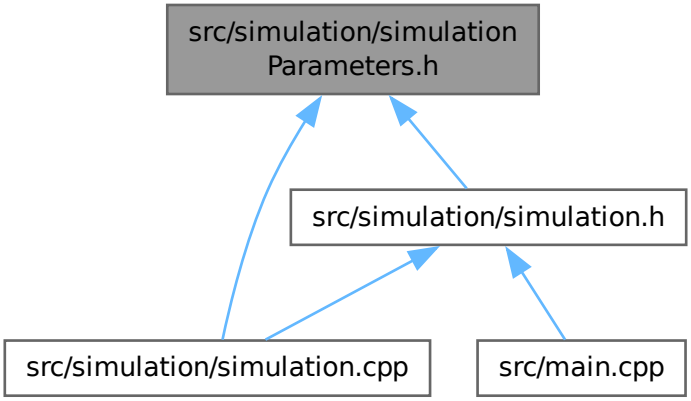
9.35 src/simulation/simulationParameters.h File Reference

```
#include <iostream>
#include <filesystem>
#include <string>
#include <vector>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
```

Include dependency graph for simulationParameters.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct SimulationParameters

9.36 simulationParameters.h

Go to the documentation of this file.

```
#ifndef SIMULATIONPARAMETERS_H
#define SIMULATIONPARAMETERS_H
#include <iostream>
#include <filesystem>
#include <string>
#include <vector>
#include <highfive/H5Easy.hpp>
#include <highfive/highfive.hpp>
// Include highfive?
//

//*********************
// lattice_dataset
// model type
// L
// dim
// J
// temp
// potts_q
// time_steps
// recording_steps
// specifying what to record
//   - magnetisation
//   - energy
//   - susceptibility
//   - specific heat
//   - correlation length
//   - correlation function


struct SimulationParameters {
    std::unique_ptr<HighFive::DataSet> lattice_set;
    std::unique_ptr<HighFive::DataSet> magnetisation_set;
    std::unique_ptr<HighFive::DataSet> energy_set;
    std::unique_ptr<HighFive::DataSet> rg_magnetisation_set1;
    std::unique_ptr<HighFive::DataSet> rg_magnetisation_set2;
    std::unique_ptr<HighFive::DataSet> rg_magnetisation_set3;
    std::unique_ptr<HighFive::DataSet> rg_energy_set1;
    std::unique_ptr<HighFive::DataSet> rg_energy_set2;
    std::unique_ptr<HighFive::DataSet> rg_energy_set3;
    std::unique_ptr<HighFive::DataSet> cluster_size_set;
    std::string model_type;
    std::filesystem::path project_folder_path;
    int L;
    int dim;
    size_t N;
    double T;
    double beta;
    double J;
    int potts_q = 1;
    // double H;
    // std::vector<double> vec_H;
    // Always records the last X steps
    size_t total_sweeps;
    // bool record_correlation_length = false;
    // bool record_correlation_function = false;
    bool rg_method = false;
};


#endif
```

Index

~Lattice
Lattice< T >, 28
~Model
Model< T >, 33
~ModelBase
ModelBase, 40

average_cluster_size
Observables, 44

beta
Model< T >, 39
SimulationParameters, 68

calculate_neighbours_table
Lattice< T >, 29
calculate_reduced_neighbours_table
Model< T >, 34
ModelBase, 41
XYModel, 73
change_spin
Model< T >, 34
ModelBase, 41
XYModel, 73
change_spin_randomly
IsingModel, 24
Model< T >, 34
PottsModel, 48
XYModel, 73
close_writing
Simulation, 54
cluster_flip_neighbours
Model< T >, 35
ModelBase, 41
XYModel, 74
cluster_size
Simulation, 67
cluster_size_set
SimulationParameters, 68
cluster_stack
Simulation, 67
compute_average_cluster_size
Simulation, 54
compute_average_energy
Model< T >, 35
compute_energy_diff_flip
IsingModel, 24
ModelBase, 41
PottsModel, 48
XYModel, 75
compute_heat_capacity
Model< T >, 35
compute_magnetisation
IsingModel, 25
ModelBase, 41
PottsModel, 49
XYModel, 75
compute_reduced_lattice
Model< T >, 36
ModelBase, 41
XYModel, 75
compute_reduced_lattice_3d
Model< T >, 36
ModelBase, 41
XYModel, 76
compute_rg_energy
Model< T >, 36
ModelBase, 42
XYModel, 76
compute_rg_spin_magnetic_term
Model< T >, 36
ModelBase, 42
XYModel, 76
compute_single_partition_function
Model< T >, 37
ModelBase, 42
compute_spin_magnetic_term
IsingModel, 25, 26
Model< T >, 37
ModelBase, 42
PottsModel, 49
XYModel, 76
compute_spin_neighbours_term
IsingModel, 26, 27
Model< T >, 37, 38
PottsModel, 50
XYModel, 77
compute_total_energy
IsingModel, 27
ModelBase, 42
PottsModel, 51
XYModel, 77
correlation_function_array
Observables, 44
correlation_length
Observables, 44

delta_function
PottsModel, 51
dim
SimulationParameters, 68
do_cluster_step
Simulation, 55
do_cluster_sweep
Simulation, 55
do_metropolis_recording_sweep
Simulation, 56
do_metropolis_step
Simulation, 56
do_metropolis_sweep
Simulation, 57

DoubleLattice, 13
DoubleLattice, 15
generate_lattice, 15
get_lattice_site, 16
get_neighbours_array, 16
doubleLattice.cpp
dvec, 79
doubleLattice.h
dvec, 80
ivec, 80
dvec
doubleLattice.cpp, 79
doubleLattice.h, 80
lattice.h, 84
observables.h, 102
potts.h, 95
simulation.cpp, 103
simulation.h, 104
xy.cpp, 97
xy.h, 98

energy_array
Observables, 44
energy_set
SimulationParameters, 69
engine
rng, 9
extract_double_vector_toml
Simulation, 57
extract_int_vector_toml
Simulation, 57

file
Simulation, 67
flip_spin
Model< T >, 38
ModelBase, 42
XYModel, 78

generate_lattice
DoubleLattice, 15
IntLattice, 20
Lattice< T >, 29
get_1d_index
Lattice< T >, 29
get_lattice
Lattice< T >, 29
get_lattice_dim
Lattice< T >, 29
get_lattice_length
Lattice< T >, 29
get_lattice_site
DoubleLattice, 16
IntLattice, 20, 21
Lattice< T >, 29, 30
get_lattice_total_length
Lattice< T >, 30
get_neighbours_array
DoubleLattice, 16
IntLattice, 21
Lattice< T >, 30
get_neighbours_table
Lattice< T >, 30
get_particle_num
Lattice< T >, 30

H
PottsModel, 52
XYModel, 78

initialise_model
Simulation, 58
initialise_writing
Simulation, 58
IntLattice, 17
generate_lattice, 20
get_lattice_site, 20, 21
get_neighbours_array, 21
IntLattice, 19
potts_q, 22
intLattice.cpp
ivec, 82
intLattice.h
ivec, 83
ising.cpp
ivec, 88
ising.h
ivec, 89
IsingModel, 22
change_spin_randomly, 24
compute_energy_diff_flip, 24
compute_magnetisation, 25
compute_spin_magnetic_term, 25, 26
compute_spin_neighbours_term, 26, 27
compute_total_energy, 27
IsingModel, 24
ivec
doubleLattice.h, 80
intLattice.cpp, 82
intLattice.h, 83
ising.cpp, 88
ising.h, 89
lattice.h, 84
model.h, 90
observables.h, 102
potts.cpp, 94
potts.h, 95
simulation.cpp, 103
simulation.h, 104
xy.cpp, 97
xy.h, 98

J
Model< T >, 39
SimulationParameters, 69

L
SimulationParameters, 69

Lattice
Lattice< T >, 28
lattice
Lattice< T >, 31
Lattice< T >, 28
~Lattice, 28
calculate_neighbours_table, 29
generate_lattice, 29
get_1d_index, 29
get_lattice, 29
get_lattice_dim, 29
get_lattice_length, 29
get_lattice_site, 29, 30
get_lattice_total_length, 30
get_neighbours_array, 30
get_neighbours_table, 30
get_particle_num, 30
Lattice, 28
lattice, 31
lattice_dim, 31
lattice_length, 31
neighbours_table, 31
particle_num, 31
lattice.h
dvec, 84
ivec, 84
lattice_dim
Lattice< T >, 31
lattice_length
Lattice< T >, 31
lattice_obj
Model< T >, 39
lattice_set
SimulationParameters, 69

magnetisation_set
SimulationParameters, 69
main
main.cpp, 87
main.cpp
main, 87
Model
Model< T >, 33
model
Simulation, 67
Model< T >, 31
~Model, 33
beta, 39
calculate_reduced_neighbours_table, 34
change_spin, 34
change_spin_randomly, 34
cluster_flip_neighbours, 35
compute_average_energy, 35
compute_heat_capacity, 35
compute_reduced_lattice, 36
compute_reduced_lattice_3d, 36
compute_rg_energy, 36
compute_rg_spin_magnetic_term, 36
compute_single_partition_function, 37
compute_spin_magnetic_term, 37
compute_spin_neighbours_term, 37, 38
flip_spin, 38
J, 39
lattice_obj, 39
Model, 33
old_lattice, 39
revert_state_of_system, 38
temp, 39
write_lattice, 38
model.h
ivec, 90
model_type
SimulationParameters, 69
ModelBase, 40
~ModelBase, 40
calculate_reduced_neighbours_table, 41
change_spin, 41
cluster_flip_neighbours, 41
compute_energy_diff_flip, 41
compute_magnetisation, 41
compute_reduced_lattice, 41
compute_reduced_lattice_3d, 41
compute_rg_energy, 42
compute_rg_spin_magnetic_term, 42
compute_single_partition_function, 42
compute_spin_magnetic_term, 42
compute_total_energy, 42
flip_spin, 42
revert_state_of_system, 43
write_lattice, 43

N
SimulationParameters, 69
neighbours_table
Lattice< T >, 31
normal_run
Simulation, 58

Observables, 43
average_cluster_size, 44
correlation_function_array, 44
correlation_length, 44
energy_array, 44
rg_energy1, 44
rg_energy2, 44
rg_energy3, 44
rg_x_magnetisation1, 44
rg_x_magnetisation2, 44
rg_x_magnetisation3, 44
rg_y_magnetisation1, 44
rg_y_magnetisation2, 45
rg_y_magnetisation3, 45
susceptibility, 45
x_magnetisation, 45
y_magnetisation, 45
observables
Simulation, 67
observables.h

dvec, 102
ivec, 102
old_lattice
Model< T >, 39

parameters
Simulation, 67
parse_parameters
Simulation, 59
particle_num
Lattice< T >, 31
PI
random.cpp, 99
potts.cpp
ivec, 94
potts.h
dvec, 95
ivec, 95
potts_q
IntLattice, 22
SimulationParameters, 69
PottsModel, 45
change_spin_randomly, 48
compute_energy_diff_flip, 48
compute_magnetisation, 49
compute_spin_magnetic_term, 49
compute_spin_neighbours_term, 50
compute_total_energy, 51
delta_function, 51
H, 52
PottsModel, 47
q, 52
vec_H, 52
project_folder_path
SimulationParameters, 69

q
PottsModel, 52

random.cpp
PI, 99
random_angle
rng, 9
random_angle_array
rng, 10
random_int_array
rng, 10
random_int_number
rng, 11
random_real_number
rng, 12
revert_state_of_system
Model< T >, 38
ModelBase, 43
rg_energy1
Observables, 44
rg_energy2
Observables, 44
rg_energy3
Observables, 44
rg_energy_set1
SimulationParameters, 69
rg_energy_set2
SimulationParameters, 70
rg_energy_set3
SimulationParameters, 70
rg_magnetisation_set1
SimulationParameters, 70
rg_magnetisation_set2
SimulationParameters, 70
rg_magnetisation_set3
SimulationParameters, 70
rg_method
SimulationParameters, 70
rg_run_2d
Simulation, 59
rg_run_3d
Simulation, 60
rg_x_magnetisation1
Observables, 44
rg_x_magnetisation2
Observables, 44
rg_x_magnetisation3
Observables, 44
rg_x_spins
XYModel, 78
rg_y_magnetisation1
Observables, 44
rg_y_magnetisation2
Observables, 45
rg_y_magnetisation3
Observables, 45
rg_y_spins
XYModel, 78
rng, 8
engine, 9
random_angle, 9
random_angle_array, 10
random_int_array, 10
random_int_number, 11
random_real_number, 12
update_seed, 12
run
Simulation, 61

Simulation, 52
close_writing, 54
cluster_size, 67
cluster_stack, 67
compute_average_cluster_size, 54
do_cluster_step, 55
do_cluster_sweep, 55
do_metropolis_recording_sweep, 56
do_metropolis_step, 56
do_metropolis_sweep, 57
extract_double_vector_toml, 57
extract_int_vector_toml, 57
file, 67

initialise_model, 58
initialise_writing, 58
model, 67
normal_run, 58
observables, 67
parameters, 67
parse_parameters, 59
rg_run_2d, 59
rg_run_3d, 60
run, 61
Simulation, 54
spins_flipped, 67
time_step, 67
update_observables, 62
update_rg_observables, 63
update_rg_observables_3d, 63
visited, 67
write_lattice, 64
write_observables_after_loop, 64
write_observables_final, 65
write_rg_observables_after_loop, 65
write_rg_observables_final, 66
simulation.cpp
dvec, 103
ivec, 103
simulation.h
dvec, 104
ivec, 104
SimulationParameters, 68
beta, 68
cluster_size_set, 68
dim, 68
energy_set, 69
J, 69
L, 69
lattice_set, 69
magnetisation_set, 69
model_type, 69
N, 69
potts_q, 69
project_folder_path, 69
rg_energy_set1, 69
rg_energy_set2, 70
rg_energy_set3, 70
rg_magnetisation_set1, 70
rg_magnetisation_set2, 70
rg_magnetisation_set3, 70
rg_method, 70
T, 70
total_sweeps, 70
spins_flipped
Simulation, 67
src Directory Reference, 8
src/lattice Directory Reference, 5
src/lattice/doubleLattice.cpp, 79
src/lattice/doubleLattice.h, 80, 81
src/lattice/intLattice.cpp, 81
src/lattice/intLattice.h, 82, 83
src/lattice/lattice.h, 83, 84
src/lattice/lattice.tpp, 85
src/main.cpp, 87
src/models Directory Reference, 6
src/models/ising.cpp, 88
src/models/ising.h, 88, 89
src/models/model.h, 90, 91
src/models/model.tpp, 92
src/models/modelBase.h, 93
src/models/potts.cpp, 94
src/models/potts.h, 95, 96
src/models/xy.cpp, 96
src/models/xy.h, 97, 98
src/random.cpp, 99
src/random.h, 99, 100
src/simulation Directory Reference, 7
src/simulation/observables.h, 101, 102
src/simulation/simulation.cpp, 102
src/simulation/simulation.h, 103, 105
src/simulation/simulationParameters.h, 106, 107
susceptibility
Observables, 45

T
SimulationParameters, 70
temp
Model< T >, 39
time_step
Simulation, 67
total_sweeps
SimulationParameters, 70

update_observables
Simulation, 62
update_rg_observables
Simulation, 63
update_rg_observables_3d
Simulation, 63
update_seed
rng, 12

vec_H
PottsModel, 52
XYModel, 79
visited
Simulation, 67

write_lattice
Model< T >, 38
ModelBase, 43
Simulation, 64
write_observables_after_loop
Simulation, 64
write_observables_final
Simulation, 65
write_rg_observables_after_loop
Simulation, 65
write_rg_observables_final
Simulation, 66

x_magnetisation
Observables, 45
xy.cpp
dvec, 97
ivec, 97
xy.h
dvec, 98
ivec, 98
XYModel, 71
calculate_reduced_neighbours_table, 73
change_spin, 73
change_spin_randomly, 73
cluster_flip_neighbours, 74
compute_energy_diff_flip, 75
compute_magnetisation, 75
compute_reduced_lattice, 75
compute_reduced_lattice_3d, 76
compute_rg_energy, 76
compute_rg_spin_magnetic_term, 76
compute_spin_magnetic_term, 76
compute_spin_neighbours_term, 77
compute_total_energy, 77
flip_spin, 78
H, 78
rg_x_spins, 78
rg_y_spins, 78
vec_H, 79
XYModel, 73

y_magnetisation
Observables, 45